

Einfluss der Entwicklerkompetenz
auf die Effektivität von AI Coding
Agents:
Eine empirische Untersuchung von
Vibe-Coding in der
Unternehmenssoftwareentwicklung

Cihad Gök

Dezember 2025

Inhaltsverzeichnis

1	Einleitung	5
1.1	Motivation	5
1.2	Problemstellung	5
1.3	Zielsetzung der Arbeit	5
1.4	Aufbau der Arbeit	5
2	Theoretischer Hintergrund und Stand der Forschung	6
2.1	AI Coding Agents und das Paradigma des Vibe-Codings	6
2.1.1	Technologische Grundlagen: Transformer und In-Context Learning	6
2.1.2	Definition: Vibe-Coding als System-1-Aktivität	7
2.1.3	Theoretische Einordnung in die HCI-Forschung	7
2.2	Kompetenzmodelle: Vom Dreyfus-Modell zu kognitiven Architekturen	8
2.2.1	Das Dreyfus-Modell der Expertise	8
2.2.2	Kritik und Erweiterung: ACT-R und Adaptive Expertise	8
2.2.3	Situated Learning und Cognitive Apprenticeship	9
2.3	Kognitionspsychologische Aspekte der Human-AI Interaction	9
2.3.1	Automation Bias und Vertrauen	9
2.3.2	Cognitive Load Theory (CLT)	9
2.3.3	Mixed-Initiative Interaction	10
2.4	Empirische Studien zu KI-Assistenz	10
2.5	Softwarequalität und Produktivität	10
2.5.1	Softwarequalität nach ISO/IEC 25010	10
2.5.2	Produktivität: Das SPACE-Framework	11
2.6	Zusammenfassung und Forschungslücke	11
3	Forschungsdesign und Hypothesenentwicklung	12
3.1	Methodischer Ansatz: Between-Subjects Design	12
3.2	Theoretische Herleitung der Hypothesen	12
3.2.1	H1: Der „Sweet Spot“ der KI-Unterstützung (Mid-Code)	12
3.2.2	H2: Das Validierungs-Paradoxon (Low-Code)	13
3.2.3	H3: Skeptizismus und Qualitätsfokus (High-Code)	13
3.3	Operationalisierung der Kompetenzstufen	13
3.4	Kontrolle von Störvariablen (Confounder)	13
3.5	Die Testumgebung als unabhängige Variable: Ecological Validity	14
4	Durchführung und Methodik der Datenerhebung	15
4.1	Ablauf der Untersuchung	15

4.2	Operationalisierung der Metriken	16
4.2.1	Funktionale Korrektheit (Effectiveness)	16
4.2.2	Effizienz (Efficiency)	16
4.2.3	Codequalität (Quality)	16
4.2.4	Interaktionsqualität (Interaction Quality)	16
4.3	Güte der Untersuchung (Threats to Validity)	17
4.3.1	Konstruktvalidität (Construct Validity)	17
4.3.2	Interne Validität (Internal Validity)	17
4.3.3	Externe Validität (External Validity)	17
4.3.4	Reliabilität (Reliability)	18
4.4	Datenanalyse-Strategie	18
5	Technische Realisierung der Testumgebung	19
5.1	Anforderungen an die Sandbox	19
5.2	Architektur und Tech-Stack: Kognitive Dimensionen	19
5.2.1	Backend: Node.js & Express (Layered Architecture)	19
5.2.2	Frontend: React & Vite	20
5.3	Technische Funktionsweise von Copilot in der Sandbox	20
5.3.1	Prompt Construction und Context Retrieval	20
5.3.2	Token Prediction und Ranking	21
5.4	Design der Aufgaben (Workload Grading)	21
5.5	Automatisierung der Evaluation	21
5.5.1	Das Analyse-Skript (<code>run.js</code>)	21
5.5.2	Grenzen der LLM-gestützten Code-Analyse	22
5.6	Relevanz für den Unternehmenskontext (BuyIn)	22
5.7	Containerisierung und Reproduzierbarkeit	22
6	Ergebnisse	23
6.1	Datengrundlage und Toolkontext	23
6.1.1	Studiensetting und Rahmenbedingungen	23
6.1.2	Datenquellen und Artefakte	23
6.1.3	Dateninventar und Vollständigkeit	24
6.1.4	Zuordnung und Zusammenführung der Daten	24
6.1.5	Reproduzierbarkeit und Auswertungswerkzeuge	24
6.1.6	Abgrenzung zu Kapitel 7	25
6.2	Task-Pipeline-Analyse	25
6.2.1	Methodische Definitionen	25
6.2.2	Pipeline-Ergebnisse (T1–T5)	26
6.2.3	Beobachtungen und Interpretation	26
6.3	Detaillierte Lösungsstrategien pro Task (T1–T5)	27
6.4	Code-Qualität und technische Metriken	31
6.4.1	ESLint und Typisierung	31
6.4.2	Testabdeckung und Teststabilität	31
6.4.3	Architektur- und Strukturindikatoren	32

6.4.4	Zusammenhänge mit Task-Fortschritt	32
6.4.5	Kurzfazit: Beobachtete Qualitätsmerkmale	32
6.5	Chat-Interaktionsmuster	33
6.5.1	Umfang und Intensität der Interaktion	33
6.5.2	Struktur der Prompts	34
6.5.3	Debugging-Interaktionen und Iterationsmuster	34
6.5.4	Modelltypische Antwortmuster (Claude Sonnet 4.5)	35
6.5.5	Beziehung zwischen Chat-Mustern und Task-Fortschritt (deskriptiv)	35
6.6	Triangulation der Datenquellen	36
6.6.1	Datenbasis und Aggregationslogik	36
6.6.2	Pipeline-Fortschritt und Implementationsstrategien (Strukturindi- katoren)	37
6.6.3	Pipeline-Fortschritt und technische Metriken	37
6.6.4	Pipeline-Fortschritt und Chat-Interaktionsmetriken	38
6.6.5	Pipeline-Fortschritt und Survey-Merkmale	38
6.6.6	Abgrenzung zu Kapitel 7	39
6.7	Zusammenfassung der Ergebnisse	39

1 Einleitung

1.1 Motivation

Die Softwareentwicklung befindet sich in einem fundamentalen Wandel. Durch den Einsatz von *Large Language Models* (LLMs) und darauf basierenden *AI Coding Agents* verändert sich die Art und Weise, wie Software konzipiert, implementiert und gewartet wird.

1.2 Problemstellung

Unternehmen stehen vor der Herausforderung, diese neuen Werkzeuge produktiv und sicher in ihre bestehenden Entwicklungsprozesse zu integrieren. Dabei ist unklar, wie sich der Einsatz von AI Coding Agents auf die Codequalität, die Entwicklerproduktivität und die Sicherheit auswirkt.

1.3 Zielsetzung der Arbeit

Ziel dieser Arbeit ist es, verschiedene AI Coding Agents zu bewerten und eine Integrationsstrategie für das Unternehmen BuyIn zu entwickeln. Dabei wird das Konzept des *Vibe-Coding* als neuer Entwicklungsstil untersucht.

1.4 Aufbau der Arbeit

Die Arbeit gliedert sich in folgende Kapitel:

- Kapitel 2: Hintergrund und Stand der Forschung
- Kapitel 3: Forschungsdesign
- Kapitel 4: Methodik
- Kapitel 5: Implementierung der Sandbox
- ...

2 Theoretischer Hintergrund und Stand der Forschung

Dieses Kapitel legt das theoretische Fundament für die Untersuchung der Interaktion zwischen menschlicher Kompetenz und KI-gestützter Code-Generierung. Es definiert zunächst das Konzept der *AI Coding Agents* und grenzt den neuartigen Arbeitsmodus des *Vibe-Codings* von klassischer Softwareentwicklung ab (Abschnitt 2.1). Darauf aufbauend wird ein Kompetenzmodell eingeführt, das auf dem Dreyfus-Modell der Expertise basiert, jedoch kritisch um moderne kognitive Architekturen wie ACT-R erweitert wird (Abschnitt 2.2). Anschließend werden kognitionspsychologische Aspekte der Mensch-Maschine-Interaktion – insbesondere *Dual-Process Theory*, *Cognitive Load* und *Automation Bias* – beleuchtet, die erklären, warum unterschiedliche Kompetenzstufen unterschiedlich stark von KI profitieren (Abschnitt 2.3). Abschließend werden Qualitäts- und Produktivitätsmodelle (ISO 25010, SPACE) vorgestellt, die als Messinstrumente dienen (Abschnitt 2.5).

2.1 AI Coding Agents und das Paradigma des Vibe-Codings

Die Integration von *Large Language Models (LLMs)* in die Softwareentwicklung markiert einen Paradigmenwechsel. Während klassische IDE-Features wie IntelliSense deterministisch und syntaktisch operieren, arbeiten AI Coding Agents probabilistisch und semantisch.

2.1.1 Technologische Grundlagen: Transformer und In-Context Learning

Moderne AI Coding Agents wie **GitHub Copilot** basieren auf der Transformer-Architektur (Vaswani u. a. 2017), die es ermöglicht, Abhängigkeiten in Sequenzen über lange Distanzen zu modellieren. Technisch relevant für die Interaktion ist das *Fill-In-The-Middle (FIM)* Paradigma (Bavarian u. a. 2022). Das Modell sagt nicht nur das nächste Token voraus, sondern berücksichtigt den Kontext *vor* und *nach* der Cursor-Position.

Zusätzlich nutzen diese Systeme *Retrieval-Augmented Generation (RAG)*-ähnliche Mechanismen. Der Agent scannt den lokalen Kontext (geöffnete Tabs, importierte Module) und injiziert relevante Code-Snippets in den Prompt, ohne dass der Nutzer dies explizit steuert. Dies reduziert die Notwendigkeit für manuelles Prompt Engineering,

erhöht aber die Abhängigkeit von einer sauberen Projektstruktur, da der Agent nur so gut ist wie der Kontext, den er „sieht“ (Witte u. a. 2023).

2.1.2 Definition: Vibe-Coding als System-1-Aktivität

Der Begriff *Vibe-Coding* beschreibt einen emergenten Entwicklungsstil, bei dem der Fokus von der imperativen Syntax-Erstellung zur deklarativen Intentions-Beschreibung verschiebt. Ich definiere Vibe-Coding für diese Arbeit als:

„Einen iterativen Prozess der Softwareerstellung, bei dem der Mensch primär als Architekt und Reviewer agiert, während die KI die Implementierungsdetails auf Basis von natürlichsprachlichen oder kontextuellen ‚Vibes‘ (Intentionen) generiert.“

Dieser Modus lässt sich durch die *Dual-Process Theory* von Kahneman (Kahneman 2011) erklären:

- **System 1 (Schnelles Denken):** Vibe-Coding appelliert an die Intuition. Der Entwickler tippt einen Funktionsnamen, der Agent schlägt den Body vor. Die Akzeptanz erfolgt oft heuristisch („Sieht gut aus“).
- **System 2 (Langsames Denken):** Die Validierung des Codes erfordert analytische Anstrengung. Hier liegt die Gefahr: Wenn der Agent (System 1) zu gut wirkt, wird System 2 nicht aktiviert, was zu Fehlern führt.

Dieser Shift verändert die Anforderungen fundamental:

- **Shift from Writing to Reading:** Der Anteil des Code-Lesens und -Verstehens steigt massiv an.
- **Shift from Syntax to Semantics:** Entwickler:innen müssen weniger wissen, *wie* eine Schleife syntaktisch korrekt ist, aber präziser verstehen, *was* sie logisch bewirken soll.

2.1.3 Theoretische Einordnung in die HCI-Forschung

In der Human-Computer Interaction (HCI) lässt sich Vibe-Coding als eine Form der *End-User Development* (EUD) verstehen, bei der die Barriere zur Code-Erstellung gesenkt wird. Es weist Parallelen zum *Natural Language Programming* auf, unterscheidet sich jedoch durch den *Mixed-Initiative*-Charakter (Horvitz 1999): Nicht nur der Mensch gibt Befehle, sondern die KI schlägt proaktiv Lösungen vor (*Ghost Text*). Dies erfordert eine ständige Synchronisation der mentalen Modelle zwischen Mensch und Maschine.

2.2 Kompetenzmodelle: Vom Dreyfus-Modell zu kognitiven Architekturen

Um den Einfluss der menschlichen Kompetenz zu messen, ist eine fundierte Klassifizierung notwendig.

2.2.1 Das Dreyfus-Modell der Expertise

Das *Dreyfus-Modell der Expertise* (H. L. Dreyfus und S. E. Dreyfus 1986) beschreibt den Erwerb von Fähigkeiten in fünf Stufen, von der strikten Regelbefolgung bis zur intuitiven Meisterschaft. Für diese Arbeit werden vier Stufen adaptiert:

1. **Novice (No-Code):** Befolgt strikte Regeln, hat kein kontextuelles Verständnis. Verlässt sich vollständig auf die KI als „Black Box“.
2. **Advanced Beginner (Low-Code):** Kann einfache Aufgaben lösen, erkennt aber keine übergeordneten Muster. Kann KI-Vorschläge syntaktisch, aber oft nicht semantisch bewerten.
3. **Competent (Mid-Code):** Arbeitet zielorientiert und kann Probleme in Teilprobleme zerlegen. Nutzt KI als Werkzeug zur Beschleunigung, besitzt aber genug Modellverständnis zur Validierung.
4. **Proficient/Expert (High-Code):** Handelt intuitiv und erkennt Abweichungen sofort. Nutzt KI zur Automatisierung von Boilerplate, ist aber skeptisch gegenüber komplexer Logik.

2.2.2 Kritik und Erweiterung: ACT-R und Adaptive Expertise

Das Dreyfus-Modell wird oft als zu linear kritisiert. Es vernachlässigt die *adaptive Expertise* (Hatano & Inagaki), also die Fähigkeit, Wissen auf neue Domänen zu übertragen. Hier bietet die kognitive Architektur **ACT-R** (Adaptive Control of Thought-Rational) von Anderson (1996) eine tiefere Erklärung:

- **Deklaratives Wissen:** Faktenwissen („Was ist eine Schleife?“). Novizen müssen dieses Wissen mühsam abrufen.
- **Prozedurales Wissen:** Handlungswissen („Wie schreibe ich eine Schleife?“). Experten haben dies in „Produktionsregeln“ kompiliert, die schnell und mühelos ablaufen.

Relevanz für Copilot: Der Agent liefert *prozedurale Chunks* (fertigen Code). Für Novizen, die nur über deklaratives Wissen verfügen, wirkt dies wie eine „Prothese“, die fehlende Produktionsregeln ersetzt. Für Experten ist es eine Beschleunigung bereits vorhandener Regeln.

2.2.3 Situated Learning und Cognitive Apprenticeship

Ergänzend bietet der Ansatz des *Situated Learning* (Lave und Wenger 1991) und *Cognitive Apprenticeship* (Collins u. a. 1989) eine Perspektive auf das Lernen mit KI.

- **Scaffolding:** Der Agent dient als „Gerüst“ (Scaffold), das es Lernenden ermöglicht, Aufgaben zu lösen, die knapp über ihrem aktuellen Kompetenzniveau liegen (Zone der nächsten Entwicklung).
- **Legitimate Peripheral Participation:** Novizen können durch KI-Unterstützung früher an produktiven Prozessen teilnehmen, auch wenn sie die volle Komplexität noch nicht durchdringen.

2.3 Kognitionspsychologische Aspekte der Human-AI Interaction

Der Erfolg von AI Coding Agents hängt maßgeblich von der menschlichen Komponente ab. Theorien der Mensch-Maschine-Interaktion erklären, warum Nutzende unterschiedlich auf KI-Vorschläge reagieren.

2.3.1 Automation Bias und Vertrauen

Ein zentrales Phänomen ist der *Automation Bias* (Parasuraman und Manzey 2010): Die Tendenz, automatisierten Systemen mehr zu vertrauen als der eigenen Urteilkraft. Dies manifestiert sich in zwei Fehlerarten:

- **Errors of Commission:** Der Nutzer akzeptiert einen falschen Vorschlag der KI (häufig bei Low-Code).
- **Errors of Omission:** Der Nutzer übersieht einen Fehler, weil das System keine Warnung ausgibt.

Das Vertrauen (*Trust*) muss kalibriert sein (Lee und See 2004). *Overtrust* führt zu unkritischer Übernahme, *Distrust* (oft bei Experten) zu ineffizienter manueller Überprüfung.

2.3.2 Cognitive Load Theory (CLT)

Die *Cognitive Load Theory* (Sweller 1988) unterscheidet drei Arten der kognitiven Belastung:

- **Intrinsic Load:** Die inhärente Komplexität der Aufgabe (z.B. Rekursion verstehen).
- **Extraneous Load:** Belastung durch die Darstellung oder das Werkzeug (z.B. komplexe IDE-Menüs).

- **Germane Load:** Die kognitive Kapazität, die für das Lernen und Schema-Bildung genutzt wird.

AI Coding Agents haben das Potenzial, den *Extraneous Load* (Syntax-Details) massiv zu senken. Dies setzt jedoch *Germane Load* frei, die für die Validierung genutzt werden muss. Wenn Novizen keine mentalen Modelle (Schemata) haben, können sie diese freigewordene Kapazität nicht nutzen, und der *Intrinsic Load* der Validierung überfordert sie. **Kritik:** Die CLT ist schwer in Echtzeit zu messen. Dennoch bietet sie ein robustes Erklärungsmodell für die Überforderung von Novizen trotz KI-Hilfe.

2.3.3 Mixed-Initiative Interaction

Die Interaktion mit Copilot ist ein Beispiel für *Mixed-Initiative Interaction* (Horvitz 1999). Beide Parteien (Mensch und Agent) können die Initiative ergreifen. Der Agent schlägt proaktiv Code vor (Ghost Text), der Mensch kann diesen annehmen, ablehnen oder durch Weitertippen modifizieren. Die Effizienz dieser Kollaboration hängt davon ab, wie gut der Mensch die Intention des Agenten vorhersagen und steuern kann (*Theory of Mind* für die KI).

2.4 Empirische Studien zu KI-Assistenz

Aktuelle Studien bestätigen die theoretischen Annahmen teilweise, zeigen aber auch neue Phänomene:

- **Grounded Copilot:** Barke u. a. (2023) identifizierten zwei Modi der Interaktion: *Acceleration* (schnelleres Tippen bekannter Logik) und *Exploration* (Nutzen der KI, um neue APIs zu entdecken).
- **Reading Between the Lines:** Mozannar u. a. (2022) zeigten, dass das Modifizieren von KI-Code oft kognitiv teurer ist als das Selbstschreiben, wenn der Vorschlag subtile Fehler enthält.

2.5 Softwarequalität und Produktivität

Die Bewertung der Arbeit mit AI Coding Agents erfordert klare Kriterien für Qualität und Produktivität.

2.5.1 Softwarequalität nach ISO/IEC 25010

Der Standard ISO/IEC 25010 definiert Softwarequalität über acht Hauptmerkmale (*ISO/IEC 25010:2011 Systems and Software Engineering—System and Software Quality Models* 2011). Für die Bewertung von KI-generiertem Code sind insbesondere relevant:

- **Funktionale Eignung:** Erfüllt der generierte Code die fachlichen Anforderungen korrekt?

- **Wartbarkeit:** Ist der Code modular, verständlich und konsistent strukturiert? Studien zeigen, dass KI-Tools dazu verleiten können, Code zu duplizieren statt zu abstrahieren, was die Wartbarkeit langfristig gefährdet (Nadi u. a. 2022).
- **Sicherheit:** Enthält der Code Schwachstellen? Da LLMs auf öffentlichen Daten trainiert sind, können sie unsichere Muster reproduzieren (Pearce u. a. 2022).

2.5.2 Produktivität: Das SPACE-Framework

Produktivität in der Softwareentwicklung ist mehrdimensional. Das SPACE-Framework (Forsgren u. a. 2021) unterscheidet fünf Dimensionen, die auch für den Einsatz von Co-pilot zentral sind:

- **S (Satisfaction):** Steigert das Tool die Zufriedenheit und reduziert es Frustration?
- **P (Performance):** Führt der Einsatz zu qualitativ besseren Ergebnissen?
- **A (Activity):** Wie viele Aufgaben können in einer bestimmten Zeit abgeschlossen werden?
- **C (Communication):** Wie verändert sich die Zusammenarbeit (hier: Mensch-Maschine-Interaktion)?
- **E (Efficiency & Flow):** Wird der Arbeitsfluss durch unterbrechungsfreie Vorschläge gefördert?

2.6 Zusammenfassung und Forschungslücke

Bisherige Studien konzentrierten sich oft auf die technische Leistungsfähigkeit von Modellen oder allgemeine Produktivitätsgewinne (Peng u. a. 2023). Es fehlt jedoch an einer differenzierten Betrachtung, wie die *individuelle Kompetenz* (operationalisiert durch das Dreyfus-Modell und ACT-R) die Interaktion mit dem Agenten moderiert. Insbesondere die Frage, ob KI als „Equalizer“ wirkt (Novizen auf Experten-Niveau hebt) oder die Kluft vergrößert („Rich get richer“), ist ungeklärt. Diese Arbeit adressiert diese Lücke durch ein experimentelles Design, das kognitive Theorien (Dual-Process, CLT) mit empirischen Software-Engineering-Metriken verbindet.

3 Forschungsdesign und Hypothesenentwicklung

Dieses Kapitel leitet das empirische Design der Studie aus den in Kapitel 2 vorgestellten Theorien ab. Ziel ist es, den Einfluss der unabhängigen Variable *Kompetenz* auf die abhängigen Variablen *Qualität*, *Effizienz* und *Interaktion* in einer kontrollierten Brownfield-Umgebung zu isolieren.

3.1 Methodischer Ansatz: Between-Subjects Design

Die Untersuchung folgt einem *Between-Subjects Design*, bei dem Teilnehmende basierend auf ihrer Vorkenntnis in disjunkte Gruppen eingeteilt werden. **Begründung nach Wohlin u. a. (2012):**

- **Vermeidung von Lerneffekten (Carry-Over Effects):** Ein Within-Subjects Design (jede Person löst Aufgaben einmal mit, einmal ohne KI) wäre hier problematisch, da das Lösen einer Aufgabe Wissen für den zweiten Durchgang generiert.
- **Isolierung der Kompetenzvariable:** Das Ziel ist der Vergleich der Interaktionsmuster zwischen verschiedenen Kompetenzniveaus („Mensch A + Maschine“ vs. „Mensch B + Maschine“), nicht der Vergleich des Tools an sich.

3.2 Theoretische Herleitung der Hypothesen

Die Hypothesen werden direkt aus dem Dreyfus-Modell, der Dual-Process Theory und der Cognitive Load Theory (siehe Abschnitt 2.2 und 2.3) abgeleitet.

3.2.1 H1: Der „Sweet Spot“ der KI-Unterstützung (Mid-Code)

Theoretische Begründung: Nach der Cognitive Load Theory profitieren Lernende am meisten, wenn *Extraneous Load* (Syntax) reduziert wird, solange genug Schema-Wissen vorhanden ist, um den Output zu validieren. Mid-Coder befinden sich in einer Phase, in der sie Konzepte verstehen, aber die syntaktische Umsetzung noch kognitive Ressourcen bindet. Copilot übernimmt diese Last, wodurch *Germane Load* für architektonische Entscheidungen frei wird. **Alternative Hypothese:** Die „Rising Tide“-Hypothese würde besagen, dass alle Gruppen gleichermaßen profitieren. Dies widerspricht jedoch der Theorie der *Zone der nächsten Entwicklung*, da Novizen ohne Scaffolding überfordert sind.

Hypothese 1. *Teilnehmende der Gruppe **Mid-Code** erzielen den höchsten relativen Produktivitätsgewinn und die ausgewogenste Balance aus Geschwindigkeit und Qualität. Sie besitzen genug Expertise, um Automation Bias zu vermeiden, profitieren aber signifikant von der syntaktischen Entlastung.*

3.2.2 H2: Das Validierungs-Paradoxon (Low-Code)

Theoretische Begründung: Novizen und Advanced Beginners (Dreyfus Stufe 1–2) fehlt das mentale Modell, um subtile Fehler zu erkennen. Sie operieren primär im *System 1* (intuitiv) und akzeptieren plausibel klingende Vorschläge unkritisch (*Errors of Commission*). Da ihnen die Schemata fehlen, um den Code zu „parsen“, ist der *Intrinsic Load* der Validierung zu hoch. **Empirische Stützung:** Studien von Vaithilingam u. a. (2022) zeigen, dass Nutzer oft Code akzeptieren, den sie nicht verstehen, was zu schwer findbaren Bugs führt.

Hypothese 2. *Teilnehmende der Gruppen **No-Code** und **Low-Code** erreichen zwar funktionale Lösungen („es läuft“), weisen aber signifikant mehr Sicherheitslücken und Wartbarkeitsprobleme auf als höhere Gruppen, da sie Code unkritisch übernehmen (Overtrust).*

3.2.3 H3: Skeptizismus und Qualitätsfokus (High-Code)

Theoretische Begründung: Experten (Dreyfus Stufe 4–5) haben starke mentale Modelle und vertrauen der Automation weniger. Sie aktivieren *System 2* (analytisch) häufiger, um Vorschläge zu prüfen. Dies führt zu einem *Distrust*-Verhalten, bei dem Zeiterparnisse durch manuelle Reviews teilweise negiert werden. **Alternative Sichtweise:** Man könnte annehmen, Experten seien am schnellsten. Die Theorie der *Expertise Reversal Effect* (Kalyuga et al.) legt jedoch nahe, dass Hilfestellungen für Experten redundant und störend sein können.

Hypothese 3. ***High-Coder** nutzen den Agenten selektiver (weniger „Accept All“), was zu einer geringeren reinen Codier-Geschwindigkeit im Vergleich zu Mid-Codern führen kann, aber die höchste Codequalität (Wartbarkeit, Sicherheit) sicherstellt.*

3.3 Operationalisierung der Kompetenzstufen

Die Einteilung erfolgt nicht willkürlich, sondern basierend auf einem Scoring-Modell (0–13 Punkte), das die Dimensionen des Dreyfus-Modells abbildet.

3.4 Kontrolle von Störvariablen (Confounder)

Um die interne Validität zu sichern, wurden folgende Störvariablen kontrolliert:

- **IDE-Familiarity:** Alle Teilnehmenden nutzten VS Code. Unterschiede in der Vertrautheit wurden im Pre-Survey erfasst.

Gruppe	Dreyfus-Level	Score	Charakteristik
No-Code	Novice	0	Kein mentales Modell von Programmierung. Verlässt sich zu 100% auf KI als Black Box.
Low-Code	Advanced Beginner	1–5	Kennt Grundkonzepte, aber keine Architekturmuster. Hohes Risiko für <i>Spaghetti-Code</i> und <i>Automation Bias</i> .
Mid-Code	Competent	6–10	Versteht Zusammenhänge, plant Schritte. Kann KI-Fehler meist erkennen (<i>System 2</i> Aktivierung möglich).
High-Code	Proficient/Expert	>10	Intuitives Verständnis. Nutzt KI zur Beschleunigung von Boilerplate, nicht zur Lösungsfindung.

Tabelle 3.1: Mapping von Kompetenzgruppen auf das Dreyfus-Modell

- **Domain Knowledge:** Das Todo-App-Szenario ist generisch genug, um kein spezifisches Domänenwissen (wie z.B. in der Medizin) vorauszusetzen.
- **Englischkenntnisse:** Da Copilot primär auf englischem Code trainiert ist, wurden Grundkenntnisse vorausgesetzt.

3.5 Die Testumgebung als unabhängige Variable: Ecological Validity

Die Wahl einer **Brownfield-Umgebung** (bestehender Code) statt Greenfield (leeres Blatt) ist essenziell für die *ökologische Validität* der Studie. In der industriellen Praxis arbeiten Entwickler:innen zu ca. 90% in bestehenden Systemen (Brandtner u. a. 2024).

- **Kontext-Sensitivität (RAG-Test):** Der Agent muss den bestehenden Style (z.B. SCSS-Module, React Hooks) erkennen und adaptieren. Dies testet die Fähigkeit des Modells, Kontext zu nutzen, und die Fähigkeit des Nutzers, diesen Kontext bereitzustellen (z.B. durch Öffnen relevanter Dateien).
- **Kognitive Belastung:** Die Teilnehmenden müssen sich in der Struktur zurechtfinden, was für Low-Coder eine zusätzliche Hürde darstellt (*Intrinsic Load*). Greenfield-Aufgaben (z.B. LeetCode) würden diese Komplexität ausblenden und die Ergebnisse verzerren.

Dies stellt sicher, dass wir *reale* Arbeitsbedingungen simulieren und nicht nur die Fähigkeit testen, isolierte Algorithmen zu generieren.

4 Durchführung und Methodik der Datenerhebung

Während das vorangegangene Kapitel das Forschungsdesign und die Hypothesen definiert hat, beschreibt dieses Kapitel den konkreten Ablauf der Studie sowie die Operationalisierung der Messgrößen. Es wird detailliert dargelegt, wie die Datenerhebung organisiert wurde, um die interne Validität der Ergebnisse sicherzustellen.

4.1 Ablauf der Untersuchung

Die Studie wurde als kontrolliertes Experiment durchgeführt. Jede Sitzung folgte einem strikten Protokoll, um Störfaktoren zu minimieren. Der Ablauf gliederte sich in fünf Phasen:

1. **Onboarding (10 Min.):** Die Teilnehmenden erhielten eine Einführung in das Szenario („Sie sind neuer Entwickler im Team...“) und die Entwicklungsumgebung. Es wurde sichergestellt, dass GitHub Copilot aktiviert und funktionsfähig war.
2. **Pre-Survey (5 Min.):** Erfassung der demografischen Daten und Selbsteinschätzung zur Kompetenzeinstufung (siehe Kapitel 3).
3. **Vibe-Coding-Challenge (60 Min.):** Die Kernphase der Untersuchung. Die Teilnehmenden bearbeiteten die Aufgaben selbstständig. Eingriffe durch den Versuchsleiter erfolgten nur bei technischen Problemen (z.B. Absturz der IDE), nicht jedoch bei inhaltlichen Fragen. **Begründung des Zeitlimits:** Die Begrenzung auf 60 Minuten dient nicht nur der organisatorischen Machbarkeit, sondern fungiert als Stressor. Zeitdruck erhöht die Wahrscheinlichkeit, dass Teilnehmende auf heuristische Strategien (*System 1*) zurückgreifen, was den *Automation Bias* verstärken kann. Dies simuliert realistische Deadline-Situationen.
4. **Post-Survey (5 Min.):** Unmittelbar nach Abschluss der Bearbeitung füllten die Teilnehmenden den Fragebogen zur subjektiven Wahrnehmung aus (NASA-TLX, TAM).
5. **Debriefing (5 Min.):** Kurzes Abschlussgespräch, um offene Fragen zu klären und qualitatives Feedback einzuholen, das im Survey nicht abgebildet werden konnte.

4.2 Operationalisierung der Metriken

Um die Hypothesen zu prüfen, wurden die abstrakten Konzepte (Qualität, Effizienz, Interaktion) in messbare Indikatoren übersetzt.

4.2.1 Funktionale Korrektheit (Effectiveness)

Die funktionale Korrektheit wird binär pro Aufgabe bewertet:

- **Erfüllt (1):** Alle Akzeptanzkriterien („Done when“) sind erfüllt, Tests bestehen.
- **Nicht erfüllt (0):** Feature fehlt, ist fehlerhaft oder Tests schlagen fehl.

Die Summe der erfüllten Aufgaben ergibt den *Functional Score* (0–8).

4.2.2 Effizienz (Efficiency)

Da die Zeit auf 60 Minuten begrenzt war, wird Effizienz primär über den Fortschritt gemessen:

$$E = \frac{\text{Anzahl erfüllter Aufgaben}}{\text{Benötigte Zeit (min)}} \quad (4.1)$$

Für Teilnehmende, die alle Aufgaben vor Ablauf der Zeit lösten, steigt der Effizienzwert entsprechend.

4.2.3 Codequalität (Quality)

Die Qualität des erzeugten Codes wird anhand statischer Analyse und manuellem Review bewertet:

- **Linting-Fehler:** Anzahl der Warnungen und Fehler durch ESLint (objektiv).
- **Wartbarkeit:** Subjektive Bewertung der Lesbarkeit und Struktur (z.B. sinnvolle Variablennamen, Modularisierung) auf einer Likert-Skala (1–5). Um die Reliabilität zu sichern, orientiert sich die Bewertung an den Clean-Code-Prinzipien von Martin (2009).

4.2.4 Interaktionsqualität (Interaction Quality)

Die Interaktion mit dem Agenten wird durch quantitative Telemetrie und qualitative Beobachtung erfasst:

- **Acceptance Rate:** Das Verhältnis von akzeptierten zu angezeigten Vorschlägen (Ghost Text). Eine hohe Rate bei Low-Codern deutet auf *Automation Bias* hin.
- **Prompt Iterations:** Wie oft muss ein Prompt umformuliert werden, bis das Ergebnis passt? Dies misst die Fähigkeit zum *Prompt Engineering*.

- **Backtracking:** Wie oft wird akzeptierter Code wieder gelöscht? Ein Indikator für *Trial & Error*.
- **Prompt-Typologie:** Unterscheidung zwischen *Inline-Kommentaren* (Code-nahe Steuerung) und *Chat-Panel* (konzeptionelle Fragen).

4.3 Güte der Untersuchung (Threats to Validity)

Die Validität der Studie wird anhand der Kategorien von Cook und Campbell (1979) diskutiert.

4.3.1 Konstruktvalidität (Construct Validity)

Messen wir wirklich das, was wir messen wollen?

- **Problem:** „Kompetenz“ ist schwer zu quantifizieren.
- **Mitigation:** Triangulation der Kompetenzmessung durch Selbsteinschätzung (Pre-Survey), objektive Berufsjahre und qualitative Beobachtung während der Aufgaben.

4.3.2 Interne Validität (Internal Validity)

Können die beobachteten Effekte eindeutig der unabhängigen Variable (Kompetenz) zugeschrieben werden?

- **Selection Bias:** Die Zuteilung zu den Gruppen basierte auf Selbsteinschätzung. Um dies zu mitigieren, wurde der Algorithmus zur Klassifizierung strikt angewendet.
- **Experimenter Bias (Rosenthal-Effekt):** Die Erwartungshaltung des Versuchsleiters könnte die Teilnehmenden beeinflussen. **Gegenmaßnahme:** Standardisierte Instruktionen und minimale Interaktion während der Coding-Phase.
- **Compensatory Rivalry (John Henry Effect):** Wenn eine Gruppe sich benachteiligt fühlt, könnte sie sich mehr anstrengen. Da alle Gruppen Copilot nutzten, ist dieses Risiko gering, jedoch könnten High-Coder versuchen, „besser als die KI“ zu sein (Saretsky 1972).

4.3.3 Externe Validität (External Validity)

Sind die Ergebnisse verallgemeinerbar?

- **Hawthorne-Effekt:** Das Wissen, beobachtet zu werden, verändert das Verhalten. Dies ist in Laborstudien unvermeidbar, wurde aber durch die realistische Aufgabenstellung (Flow-Erleben) abgemildert.

- **Ecological Validity:** Die Nutzung einer Brownfield-Umgebung erhöht die Übertragbarkeit auf Unternehmenskontexte im Vergleich zu synthetischen LeetCode-Aufgaben massiv.

4.3.4 Reliabilität (Reliability)

Sind die Ergebnisse reproduzierbar?

- **Inter-Rater Reliability:** Die Bewertung der Codequalität (Wartbarkeit) unterliegt einer gewissen Subjektivität. Durch die Verwendung eines standardisierten Kriterienkatalogs (basierend auf ISO 25010) wird versucht, die Varianz zu minimieren.

4.4 Datenanalyse-Strategie

Die Auswertung erfolgt mittels eines *Mixed-Methods*-Ansatzes:

1. **Quantitative Analyse:** Vergleich der Mittelwerte von Functional Score und Effizienz zwischen den vier Gruppen. Aufgrund der kleinen Stichprobe werden deskriptive Statistiken und Visualisierungen (Boxplots) bevorzugt.
2. **Qualitative Analyse:** Inhaltsanalyse der Screen-Recordings und des Codes, um Verhaltensmuster zu identifizieren (z.B. „Trial & Error“ bei Low-Codern vs. „Review & Refine“ bei High-Codern).

Dieser methodische Aufbau stellt sicher, dass nicht nur das *Ergebnis* (Code), sondern auch der *Prozess* (Interaktion) beleuchtet wird.

5 Technische Realisierung der Testumgebung

Dieses Kapitel dokumentiert die technische Implementierung der *BuyIn Todo Sandbox*, die als experimentelle Plattform für diese Studie dient. Es wird erläutert, wie die Brownfield-Umgebung architektonisch aufgebaut ist, um realistische Unternehmensanforderungen zu simulieren, und welche technischen Maßnahmen ergriffen wurden, um eine reproduzierbare Datenerhebung zu gewährleisten.

5.1 Anforderungen an die Sandbox

Um valide Aussagen über Vibe-Coding in Unternehmenskontexten treffen zu können, musste die Testumgebung folgende Anforderungen erfüllen:

1. **Realismus (Brownfield):** Die Codebasis darf nicht leer sein („Greenfield“), sondern muss bestehende Strukturen, technische Schulden und Konventionen enthalten, die der Agent und der Mensch verstehen müssen.
2. **Komplexität:** Der Tech-Stack muss modern, aber komplex genug sein, um triviale Lösungen zu verhindern.
3. **Isolierbarkeit:** Die Umgebung muss für jeden Teilnehmenden identisch zurückgesetzt werden können (Containerisierung).
4. **Messbarkeit:** Das System muss automatisierte Tests bereitstellen, um den funktionalen Fortschritt objektiv zu messen.

5.2 Architektur und Tech-Stack: Kognitive Dimensionen

Die Sandbox ist als klassische Full-Stack-Webanwendung konzipiert. Die Architekturwahl hat direkte Auswirkungen auf die kognitive Belastung der Teilnehmenden, analysiert durch das Framework der *Cognitive Dimensions of Notation* (Green 1989).

5.2.1 Backend: Node.js & Express (Layered Architecture)

Das Backend basiert auf Node.js mit dem Express-Framework und folgt einer strikten Schichtenarchitektur:

- **Routes:** Definition der API-Endpunkte.
- **Controllers:** Verarbeitung der HTTP-Requests und Validierung.
- **Services:** Kapselung der Geschäftslogik.
- **Repositories:** Datenzugriffsschicht.

Kognitive Dimension - Viscosity (Zähigkeit): Diese Trennung erhöht die *Viscosity* für Novizen, da eine Änderung an mehreren Stellen (Datei-Hopping) durchgeführt werden muss. Für Copilot hingegen ist diese Struktur vorteilhaft, da sie klare semantische Grenzen zieht, die das Modell über den Dateinamen und Pfadkontext erkennen kann.

5.2.2 Frontend: React & Vite

Das Frontend nutzt React mit TypeScript. Die Komponenten sind funktional aufgebaut und nutzen Hooks. **Kognitive Dimension - Hidden Dependencies:** Die Verwendung von *Custom Hooks* und *Context API* erzeugt *Hidden Dependencies*. Ein Low-Coder sieht im UI-Code nicht sofort, woher die Daten kommen. Dies testet die Fähigkeit des Agenten, diese Abhängigkeiten aufzulösen, und die Fähigkeit des Nutzers, dem Agenten den richtigen Kontext zu geben.

5.3 Technische Funktionsweise von Copilot in der Sandbox

Um zu verstehen, warum die Sandbox Copilot fordert, muss die technische Funktionsweise betrachtet werden.

5.3.1 Prompt Construction und Context Retrieval

Copilot sendet nicht nur den Code vor dem Cursor an das Modell. Der *Copilot Proxy* in der IDE sammelt Kontext aus:

- **Jaccard Similarity:** Offene Tabs werden nach Ähnlichkeit zum aktuellen Code gescannt.
- **Semantic Search:** Vektorisierte Suche im Projekt (bei neueren Versionen).

In der Sandbox sind die Aufgaben so gestaltet, dass die Lösung oft in einer *anderen* Datei liegt (z.B. muss der Controller angepasst werden, um den Service zu nutzen). Wenn der Nutzer diese Datei nicht öffnet, fehlt dem Modell der Kontext. Dies misst indirekt die Kompetenz des Nutzers, dem Agenten die richtigen „Hinweise“ zu geben.

5.3.2 Token Prediction und Ranking

Das Modell generiert mehrere Vorschläge (Beams) und rankt diese nach Wahrscheinlichkeit. In einer Brownfield-Umgebung ist die Wahrscheinlichkeit für „korrekten“ Code höher, wenn er dem Stil der Umgebung entspricht. Ein High-Coder erkennt, ob der Vorschlag den Stil bricht (z.B. `var` statt `const`), ein Low-Coder nicht.

5.4 Design der Aufgaben (Workload Grading)

Die Aufgaben sind so gestaffelt, dass sie die kognitive Last sukzessive erhöhen:

Task	Beschreibung	Kognitive Anforderung
1	Persistenz (JSON)	Verstehen des Repository-Patterns (Backend).
2	User-Accounts	Einführung neuer Entitäten, Anpassung mehrerer Layer.
3	Kategorien (UI)	Frontend-Logik, State-Management.
4	Attachments	Umgang mit Binärdaten, API-Erweiterung.
5	Kalender	Komplexe Logik, Datumsberechnung (hoher Intrinsic Load).
6	Email-Verifikation	Integration externer Konzepte (Mocking).
7	Performance	Optimierung, kein funktionales Feature (Experten-Task).
8	ICS-Export	Standard-Format, reines Mapping (Erholung).

Tabelle 5.1: Aufgaben-Design und kognitive Anforderungen

5.5 Automatisierung der Evaluation

Um die Auswertung der Ergebnisse zu standardisieren, wurde ein Evaluations-Framework entwickelt.

5.5.1 Das Analyse-Skript (`run.js`)

Ein Node.js-Skript automatisiert die Prüfung der Lösungen. Es führt folgende Schritte aus:

1. **Test-Execution:** Ausführung der Unit-Tests (Vitest) und End-to-End-Tests (Playwright).
2. **Linting:** Prüfung auf Code-Style-Verletzungen mittels ESLint.
3. **Report-Generierung:** Zusammenfassung der Ergebnisse in einer JSON-Datei.

5.5.2 Grenzen der LLM-gestützten Code-Analyse

Zusätzlich zur rein funktionalen Prüfung wird ein LLM-Prompt-Template verwendet, um qualitative Aspekte des Codes zu bewerten. Hierbei muss kritisch angemerkt werden, dass LLMs zu *Halluzinationen* neigen können, also Fehler finden, die nicht existieren, oder echte Probleme übersehen. Daher dient dieser Score nur als Indikator und wird stichprobenartig manuell validiert.

5.6 Relevanz für den Unternehmenskontext (BuyIn)

Die gewählte Architektur und Methodik spiegelt reale Herausforderungen bei BuyIn wider:

- **Legacy-Integration:** Wie bei BuyIn müssen Entwickler oft in gewachsenen Strukturen arbeiten, die nicht perfekt dokumentiert sind. Vibe-Coding kann hier helfen, den „impliziten Kontext“ schneller zu erschließen.
- **Security & Compliance:** Der Einsatz von Copilot in Enterprise-Umgebungen birgt Risiken (Datenabfluss, Lizenzverletzungen). Die Sandbox simuliert dies durch Aufgaben, bei denen sensible Daten (User-Accounts) behandelt werden müssen.
- **Skalierbarkeit:** Die modulare Struktur der Sandbox zeigt, wie Copilot in größeren Teams (wo Modularisierung Pflicht ist) performt, im Gegensatz zu kleinen Skript-Projekten.

5.7 Containerisierung und Reproduzierbarkeit

Die gesamte Umgebung ist mittels Docker containerisiert. Dies stellt sicher, dass alle Teilnehmenden – unabhängig von ihrem Betriebssystem – unter identischen Bedingungen arbeiten und technische Störfaktoren (z.B. unterschiedliche Node-Versionen) eliminiert werden.

6 Ergebnisse

6.1 Datengrundlage und Toolkontext

Dieses Unterkapitel beschreibt die Datengrundlage der Untersuchung sowie den technischen Kontext der durchgeführten *VibeCoding-Challenge*. Die Analyse basiert auf einem Datensatz von $N = 18$ Probanden, die im Rahmen einer kontrollierten Coding-Challenge Aufgaben zur Entwicklung einer Todo-Applikation bearbeiteten.

6.1.1 Studiensetting und Rahmenbedingungen

Die Studie wurde unter einheitlichen Rahmenbedingungen durchgeführt, um die Vergleichbarkeit der Ergebnisse zu gewährleisten:

- **Zeitlimit:** Alle Probanden hatten ein striktes Zeitlimit von 60 Minuten für die Bearbeitung der Aufgaben.
- **Tooling:** Als Entwicklungsumgebung diente Visual Studio Code mit der GitHub Copilot Extension.
- **KI-Modell:** Als zugrundeliegendes Large Language Model (LLM) wurde **Claude 3.5 Sonnet** verwendet. Die Interaktion erfolgte über das Chat-Interface von GitHub Copilot, wobei das Modell explizit als Backend konfiguriert war.
- **Aufgabenstellung:** Die Probanden erhielten eine Liste von 8 Aufgaben (T1–T8) mit steigender Komplexität, beginnend bei Backend-Persistenz bis hin zu komplexen Frontend-Features und Export-Funktionen.

6.1.2 Datenquellen und Artefakte

Die empirische Auswertung stützt sich auf drei primäre Datenquellen, die eine Triangulation der Ergebnisse ermöglichen:

1. **Code-Artefakte (Git-Repositories):** Die finalen Implementierungen der Probanden liegen als separate Git-Branches vor. Diese Branches enthalten den vollständigen Quellcode zum Zeitpunkt des Zeitablaufs.
 - Pfad: Remote-Branches mit dem Suffix `*-vibe` (z.B. `origin/august-martin-vibe`).
2. **Chat-Logs:** Die vollständigen Interaktionsprotokolle zwischen Proband und KI-Assistent wurden exportiert. Diese Protokolle enthalten alle Prompts der Nutzer sowie die Antworten und Code-Generierungen der KI.

- Pfad: `thesis/chats/` (Format: `.docx`).
3. **Survey-Daten:** Im Anschluss an die Challenge füllten die Probanden einen Fragebogen zur Selbsteinschätzung und subjektiven Wahrnehmung der KI-Interaktion aus.
- Pfad: `thesis/surveys/Probanden-Einstufungsformular (Responses).xlsx`.

6.1.3 Dateninventar und Vollständigkeit

Tabelle 6.1 zeigt das Inventar der verfügbaren Datenartefakte. Insgesamt liegen für 18 Probanden Daten vor.

Artefakt-Typ	Anzahl	Anmerkung
Git-Banches (*-vibe)	16	2 Probanden arbeiteten lokal/ohne Push*
Chat-Logs	18	Vollständig für alle Teilnehmer
Survey-Einträge	18	Vollständig ($N = 18$)

Tabelle 6.1: Inventar der Datenartefakte. (*Für die fehlenden Remote-Banches liegen lokale Stände oder Chat-Rekonstruktionen vor, die für die Analyse genutzt wurden.)

6.1.4 Zuordnung und Zusammenführung der Daten

Die Zusammenführung der Datenquellen erfolgte über eine Mapping-Tabelle, die die Klarnamen aus dem Survey und den Chat-Dateinamen den entsprechenden Git-Banches zuordnet.

- **Survey ↔ Branch:** Das Survey-Feld "Branch-Name" sowie der Name des Probanden dienten als Schlüssel.
- **Chat ↔ Branch:** Die Dateinamen der Chat-Logs (z.B. `Max_Helling.docx`) wurden normalisiert und den entsprechenden Branches (z.B. `helling-max-vibe`) zugeordnet.

Die Survey-Daten enthalten zudem eine Einteilung in Kompetenzgruppen (No-Code, Low-Code, Mid-Code, High-Code), die in der späteren Analyse als vergleichendes Merkmal herangezogen wird.

6.1.5 Reproduzierbarkeit und Auswertungswerkzeuge

Um die Ergebnisse objektiv und reproduzierbar zu gestalten, kommen automatisierte Analysewerkzeuge zum Einsatz, deren Resultate im Ordner `evaluation/` persistiert werden:

- **Statische Code-Analyse:** Einsatz von ESLint zur Überprüfung der Code-Qualität und Einhaltung von Standards.
- **Automatisierte Tests:** Ausführung der vorhandenen Test-Suite (basierend auf Jest/Vitest), um die funktionale Korrektheit der Implementierungen (Task Completion) zu verifizieren.
- **Chat-Parsing:** Algorithmische Auswertung der Chat-Logs zur Bestimmung von AttemptedTasks und Identifikation von Prompting-Mustern.

6.1.6 Abgrenzung zu Kapitel 7

Das vorliegende Kapitel 6 beschränkt sich auf die deskriptive Darstellung der Ergebnisse und die objektive Auswertung der Daten. Eine Interpretation dieser Ergebnisse, die Diskussion von Ursache-Wirkungs-Beziehungen sowie die Beantwortung der Forschungsfragen erfolgen im anschließenden Kapitel 7 (Diskussion).

6.2 Task-Pipeline-Analyse

In diesem Abschnitt wird der Fortschritt der Probanden entlang der Aufgaben-Pipeline (T1–T5) dargestellt. Ziel ist es, den *Drop-off* über die Aufgaben hinweg sichtbar zu machen und zwischen Intention (Chat), Umsetzung (Code) und testbasierter Bestätigung (Integrationstests) zu unterscheiden.

6.2.1 Methodische Definitionen

Zur Einordnung werden drei Statuskategorien verwendet:

- **Attempted (Versucht):** Ein Task gilt als *Attempted*, wenn er im Chat-Verlauf explizit als Ziel, Anforderung oder nächster Arbeitsschritt formuliert wurde. Die bloße Existenz von Dateien ohne entsprechenden Kontext im Chat genügt nicht.
- **Implemented (Umgesetzt):** Ein Task gilt als *Implemented*, wenn taskspezifische Logik nachweisbar ist – entweder durch statische Codeanalyse *oder* durch konsistentes, wiederholbares, taskspezifisches API-Verhalten, das durch Integrationstests bestätigt wird.
- **Verified (Testbasiert bestätigt):** Ein Task gilt als *Verified*, wenn die zugehörigen automatisierten Integrationstests erfolgreich durchlaufen wurden.

Hierarchie der Metriken. Die drei Metriken sind bewusst als Trichter modelliert: *Attempted* $\rightarrow$ *Implemented* $\rightarrow$ *Verified*. Durch die obige Definition gilt konzeptionell *Verified* $\subseteq$ *Implemented*: Besteht ein taskspezifischer Integrationstest, wird das bestätigte Verhalten als Implementationsnachweis gewertet. *Verified* ist damit keine übergeordnete Qualitätsaussage, sondern ein nachrangiges Diagnose-Signal innerhalb des Trichters.

Insbesondere wenn *Implemented* = 0, kann ein *Verified*-Signal (falls es dennoch auftreten würde) nicht als fachlicher Erfolg interpretiert werden, sondern wäre als Messfehler (z.B. zu tolerante Tests) zu behandeln.

Hinweis zur Testmethodik (V2). Für die Pipeline-Auswertung wurden die Tests (insbesondere T2, sowie in Teilen T3–T5) als API-Integrationstests (*Supertest* gegen Express) gestaltet, um weniger abhängig von internen Architekturentscheidungen (z.B. Service-/Repository-Struktur) zu sein. *Verified* bedeutet in dieser Studie daher explizit: „besteht die aktuellen Tests“ – nicht „fachlich vollständig korrekt gelöst“. *Verified* darf folglich nie isoliert interpretiert werden, sondern immer im Verhältnis zu *Attempted* und *Implemented*.

6.2.2 Pipeline-Ergebnisse (T1–T5)

Tabelle 6.2 fasst den Status der Aufgaben T1 bis T5 für alle $N = 18$ Probanden zusammen. Der Fokus liegt aufgrund des strikten Zeitlimits (60 Minuten) auf diesen frühen Aufgaben.

extbfTask	Attempted (n)	Implemented (n)	Verified (n)
T1: Persistenz	18 (100%)	17 (94%)	10 (56%)
T2: Auth	18 (100%)	17 (94%)	4 (22%)
T3: Kategorien	15 (83%)	13 (72%)	13 (72%)
T4: Attachments	13 (72%)	8 (44%)	0 (0%)
T5: Kalender	13 (72%)	0 (0%)	0 (0%)

Tabelle 6.2: Task-Pipeline: Von der Intention zur verifizierten Lösung ($N = 18$), basierend auf V3-Integrationstests.

6.2.3 Beobachtungen und Interpretation

Für T3–T5 ist der *Drop-off* in erster Linie zeitgetrieben: Unter dem 60-Minuten-Limit werden Integrationsaufgaben häufiger begonnen (*Attempted*), aber seltener konsistent fertiggestellt (*Implemented*) und noch seltener testkonform stabilisiert (*Verified*). „Nicht implementiert“ bedeutet dabei nicht „nicht verstanden“, sondern häufig „nicht mehr rechtzeitig integriert, refaktoriert oder debuggt“.

T1 (Persistenz). Alle Probanden adressierten Persistenz im Chat (100%), und die hohe Implementierungsrate (94%) zeigt, dass Persistenz als Kernanforderung verstanden wurde. Die Verifikationsrate (56%) weist darauf hin, dass eine robuste, testkonforme Persistenz (z.B. stabiler CRUD-Fluss, konsistente IDs, fehlerfreie Serialisierung) innerhalb des Zeitfensters nicht in allen Fällen erreicht wurde.

T2 (User Accounts/Auth). Auch T2 wurde von allen Probanden versucht (100%) und sehr häufig implementiert (94%). Die vergleichsweise niedrige *Verified*-Rate (22%) ist weniger als Qualitätsurteil zu lesen, sondern als Diagnose-Signal: Auth-Implementierungen

sind stark architektur- und konfigurationsabhängig (Endpunktpfade, Token- vs. Cookie-Mechanismen, Zugriffskontrollen), wodurch selbst funktionierende Lösungen an den konkreten Testannahmen scheitern können.

T3 (Kategorien). Der Drop-off (83% *Attempted* vs. 72% *Implemented*) deutet darauf hin, dass Kategorien häufig begonnen werden, aber nicht in allen Fällen bis zur stabilen, wiederholbaren Integration geführt werden. Durch die erweiterte *Implemented*-Definition (statischer Nachweis *oder* reproduzierbares, testspezifisch bestätigtes API-Verhalten) fällt *Verified* hier nicht als „besser“ aus, sondern bestätigt den Implementationsnachweis auf Verhaltensebene. Gleichzeitig bleibt *Verified* als Diagnosesignal relevant: Es zeigt, ob die beobachtete Kategorie-Funktionalität die konkreten Testannahmen konsistent erfüllt.

T4 (Attachments). Trotz 72% *Attempted* und 44% *Implemented* liegt *Verified* bei 0%. Das spricht für hohe Varianz in der Umsetzung (unterschiedliche Upload-Mechanismen, Feldnamen, Storage-Strategien) und dafür, dass die Testannahmen die Heterogenität nicht ausreichend abdecken. In diesem Fall ist *Verified* vor allem als Hinweis auf Test-/Schnittstelleninkonsistenz zu lesen, nicht als Beleg fehlender Kompetenz.

T5 (Kalender/Multi-Day). T5 wurde häufig begonnen (72% *Attempted*), jedoch liegt im (V3-)Pipeline-Lauf kein reproduzierbares, taskspezifisches API-Verhalten vor, das die Kalender-/Multi-Day-Semantik zuverlässig bestätigt (0% *Verified*). Entsprechend bleibt *Implemented* hier bei 0%: Weder konnte taskspezifische Logik statisch identifiziert werden, noch wurde sie durch konsistente Tests bestätigt. Inhaltlich ist dies mit dem Zeitlimit vereinbar: Kalenderlogik erfordert typischerweise zusätzliche Felder (Range), Validierung und Query-Semantik (Overlap), die unter 60 Minuten häufig nicht mehr end-to-end stabilisiert werden.

Zwischenfazit. Insgesamt zeigt die Pipeline den erwartbaren Trichter: Alle Teilnehmer starten stark bei T1/T2, während komplexere Integrationsaufgaben (T3–T5) unter Zeitdruck seltener vollständig umgesetzt werden. *Verified*-Werte werden in dieser Arbeit nicht „gefeiert“, sondern als Diagnose-Signal genutzt – für Testqualität, Architekturdiversität sowie die Differenz zwischen beobachtbarem Verhalten und tatsächlich implementierter Fachlogik.

Methodischer Hinweis zur Interpretation

Die Pipeline dient der Strukturanalyse von Arbeitsverläufen (Starten, Umsetzen, Stabilisieren) und damit der Beschreibung typischer Engpässe unter Zeitdruck – nicht der Bewertung individueller Leistung.

6.3 Detaillierte Lösungsstrategien pro Task (T1–T5)

Dieses Unterkapitel beschreibt typische Lösungsstrategien, die in den $N = 18$ *-vibe-Banches für die Tasks T1–T5 beobachtet wurden. Grundlage sind Repository-Struktur und Implementationsartefakte (Backend/Frontend, Datenhaltung), Chat-basierte *Attempted*-Signale sowie Survey-Angaben (aggregiert, ohne Klarnamen). Die Darstellung ist rein deskriptiv.

Einordnung der Datenbasis (kurz)

T1 und T2 wurden im Chat durchgängig adressiert (18/18), T3 in 15/18 sowie T4/T5 in jeweils 13/18 Fällen; die Konfidenz sinkt dabei von T1/T2 zu T4/T5. In den Surveys werden Zeitdruck und Debugging am häufigsten als Hürden genannt (je 5/18), gefolgt von Aufgabenverständnis (3/18); die Selbsteinstufung verteilt sich über No-Code (5), Low-Code (4), Mid-Code (5) und High-Coder (4).

T1 – Persistente Todos

1. **Ziel des Tasks (kurz, technisch).** Persistente Speicherung von Todos über Prozess-Neustarts hinweg, inkl. stabilem CRUD-Verhalten (Erstellen, Lesen, Aktualisieren, Löschen) und konsistenter Identifikatoren.
2. **Typische Implementierungsstrategien (mit Häufigkeiten).** Die dominante Strategie war SQLite-basierte Persistenz (14/18). Daneben traten MongoDB (1/18) und PostgreSQL (1/18) sowie verbleibende Varianten wie In-Memory/sonstige Ablagen (2/18) auf. In der Code-Struktur zeigt sich häufig ein Repository-Pattern (z.B. `TodoRepository`) mit einer klaren Abgrenzung zwischen Controller/Route und Datenzugriff.
3. **Qualitätsunterschiede (robust vs. fragil).** Robust sind konsistente ID-Schemata und deterministisches CRUD; fragil sind v.a. uneinheitliche ID-Felder (`id` vs. `_id`) und inkonsistente Update-Semantik.
4. **Typische KI-generierte Muster.** Häufig sind Repository-/Controller-Boilerplates und generische CRUD-Endpunkte, bei denen Randfälle (Validierung, Fehlercodes, ID-Normalisierung) nur teilweise konsistent umgesetzt sind.
5. **Warum der Task häufig abgebrochen oder nur teilweise umgesetzt wurde.** T1 wurde selten abgebrochen, aber häufig nur bis zu einem Minimal-CRUD stabilisiert; Zeitdruck und Debugging (DB-Setup/Build) begrenzen die Absicherung.

T2 – Benutzerkonten / Authentifizierung

1. **Ziel des Tasks (kurz, technisch).** Nutzerregistrierung und Login mit Zugriffskontrolle auf geschützte Endpunkte (z.B. `GET /api/todos`) sowie konsistenter Authentifizierungsweitergabe (Token oder Session).
2. **Typische Implementierungsstrategien (mit Häufigkeiten).** JWT ist dominant (11/18), gefolgt von Session/Cookie (4/18) und Hybrid-Ansätzen (1/18); selten sind Sonderfälle (z.B. Hashing ohne klaren Token-Flow: 1/18) bzw. fehlende/unklare Signale (1/18). Häufige Strukturmuster sind zentrale Auth-Middleware oder Prüfungen direkt im Controller.

3. **Qualitätsunterschiede (robust vs. fragil).** Robust sind eindeutige Token-/Session-Flows und konsistente Route-Guards; fragil sind inkonsistente Header-/Cookie-Nutzung und divergierende Erwartungen zwischen Frontend und Backend.
4. **Typische KI-generierte Muster.** Boilerplate-Flows (Register/Login/Protect) treten häufig auf, teils mit Over-Engineering (Rollen/Claims) oder mit Minimalismus (fehlende Validierung/Expiry); verbreitet sind Tutorial-nahe Copy-Paste-Muster (JWT + bcrypt).
5. **Warum der Task häufig abgebrochen oder nur teilweise umgesetzt wurde.** T2 ist stark schnittstellen- und konfigurationssensitiv (Endpunkte, Token-Feldnamen, Cookie-Flags) und wird laut Survey häufig durch Zeitdruck/Debugging begrenzt; die testnahe Absicherung bleibt oft nachrangig.

T3 – Kategorien

1. **Ziel des Tasks (kurz, technisch).** Erweiterung des Todo-Datenmodells um Kategorisierung und Bereitstellung über API und UI, sodass Todos kategoriebasiert erstellt, gespeichert und angezeigt werden können.
2. **Typische Implementierungsstrategien (mit Häufigkeiten).** Ein separates Kategorie-Konzept (Model/Tabelle/Collection mit Relation via `categoryId`) dominiert (14/18). Inline-Kategorien als String sind selten (1/18); in 3/18 Branches ist das Kategorie-Konzept in Struktursignalen nicht klar erkennbar.
3. **Qualitätsunterschiede (robust vs. fragil).** Robust sind konsistente Persistenz und stabile Referenzen (IDs/Relation); fragil sind rein UI-seitige Kategorien oder inkonsistente Relation/Serialisierung.
4. **Typische KI-generierte Muster.** Häufig sind zusätzliche `Category`-Modelle und Routen (teils mit unnötiger CRUD-Komplexität) sowie „Tags vs. Categories“-Mischformen, bei denen Begriffe variieren, die Datenrepräsentation aber nicht konsistent vereinheitlicht wird.
5. **Warum der Task häufig abgebrochen oder nur teilweise umgesetzt wurde.** T3 erhöht den Integrationsaufwand (Schema, UI, Backward-Compatibility); unter Zeitdruck bleibt häufig die End-to-End-Stabilisierung (Persistenz/Anzeige/Tests) unvollständig.

T4 – Dateianhänge (Attachments)

1. **Ziel des Tasks (kurz, technisch).** Upload und Zuordnung von Dateien zu Todos (typischerweise `multipart/form-data`), Speicherung (Dateisystem oder DB) und Rückgabe verwertbarer Metadaten über die API.

2. **Typische Implementierungsstrategien (mit Häufigkeiten).** In 8/18 Branches ist ein Multer-/Upload-Ansatz erkennbar; in 10/18 Branches sind Attachments nicht oder nur indirekt/unklar umgesetzt. Wo Upload existiert, ist Storage häufig lokal (Uploads-Ordner) oder metadatenbasiert (Dateiname/URL).
3. **Qualitätsunterschiede (robust vs. fragil).** Robust sind klare Upload-Verträge (Feldnamen/Metadaten) und stabile Zuordnung zum Todo; fragil sind uneinheitliche Response-Formate und fehlende Metadaten-Persistenz.
4. **Typische KI-generierte Muster.** Multer-Templates mit Boilerplate-Routen sind häufig; daneben treten Over-Engineering (komplexe Attachment-Entitäten) oder Minimal-Uploads ohne stabilen Metadaten-Rückkanal auf.
5. **Warum der Task häufig abgebrochen oder nur teilweise umgesetzt wurde.** T4 bündelt viele Fehlerquellen (multipart, Storage, Pfade/URLs, Frontend-Integration) und wird in Surveys häufig als Debugging-getrieben beschrieben; unter Zeitdruck wird es oft hinter Persistenz/Auth priorisiert.

T5 – Kalender / Multi-Day

1. **Ziel des Tasks (kurz, technisch).** Abbildung von Todos im Kalender, inklusive Unterstützung von Zeiträumen (Multi-Day), sowie konsistente Query-/Filter-Semantik (z.B. Overlap-Logik) zur effizienten Darstellung in Kalenderansichten.
2. **Typische Implementierungsstrategien (mit Häufigkeiten).** In den Branches sind Range-Felder als Struktursignale verbreitet (z.B. `dueEndDate` neben `dueDate`). Gleichzeitig zeigen die Integrationstests (V3) für T5 keine reproduzierbare End-to-End-Semantik (0/18 verifiziert), was auf eine häufige Teilstrategie hindeutet: Felder/Komponenten werden vorbereitet, die vollständige Backend-Semantik (Overlap/Filter, stabiler API-Vertrag) bleibt jedoch unvollständig.
3. **Qualitätsunterschiede (robust vs. fragil).** Robust wäre Range-Validierung (Ende > Start), zuverlässige Persistenz und klare Overlap-Filterung; fragil sind reine Feld-Präsenz ohne konsistente Query-/Filter-Semantik.
4. **Typische KI-generierte Muster.** Häufig sind UI-/Modell-Boilerplates (zusätzliche Datumsfelder, Kalender-Komponenten) sowie Copy-Paste von Range-Utilities ohne konsistenten API-Vertrag (Parameter/Overlap-Regel).
5. **Warum der Task häufig abgebrochen oder nur teilweise umgesetzt wurde.** T5 ist stark gekoppelt (Modell → Persistenz → Query → UI) und wird laut Chat/Survey häufig durch Zeitdruck und Debugging limitiert; unter 60 Minuten bleibt die testbare Semantik (Overlap/Filter, konsistente API) oft unvollständig.

6.4 Code-Qualität und technische Metriken

Dieses Unterkapitel beschreibt beobachtbare Aspekte der Code-Qualität über alle 18 *-vibe-Banches hinweg. Im Fokus stehen technische Messwerte und Artefakte (Linting, Typisierungs-Surrogate, Teststabilität und Strukturindikatoren), die die Wartbarkeit und Robustheit des Systems ergänzend zur rein funktionalen Korrektheit beschreiben. Die Darstellung bleibt bewusst deskriptiv; sie bildet Messstände und Auswertungsergebnisse ab, ohne Ursachen oder Wirkzusammenhänge abzuleiten. Als Datenbasis dienen ausschließlich die Auswertungsartefakte `evaluation/code_quality_metrics.json`, `evaluation/data_safe/lint_results.json`, `evaluation/data_safe/test_results.json`, `evaluation/deep_code_analysis.json` und `evaluation/task_pipeline_v3.json`.

6.4.1 ESLint und Typisierung

Die ESLint-Ergebnisse liegen als Fehleranzahl pro Branch vor. In `evaluation/data_safe/lint_results.json` weisen 4 von 18 Branches 0 ESLint-Fehler aus, während 14 von 18 Branches 1 ESLint-Fehler ausweisen. Eine Aufschlüsselung nach ESLint-Regelkategorien (z. B. *no-unused-vars*, *no-explicit-any*) ist in den Artefakten nicht enthalten und wurde daher nicht ausgewertet.

Als Surrogat für Typisierungsstrenge und potenziellen “Escape” aus dem Typsystem wird die Nutzung von `any` ausgewiesen (`anyUsage` in `evaluation/code_quality_metrics.json`). Über alle Branches liegt `anyUsage` zwischen 2 und 42 (Median: 8.5). Zusätzlich wird die Verwendung von `console` als Indikator für Debug-Ausgaben berichtet (`consoleUsage`); diese liegt zwischen 4 und 19 (Median: 6). Spezifische Vorkommen von TODO/FIXME oder “temporären” Debug-Endpunkten werden in den vorliegenden Metriken nicht erfasst.

6.4.2 Testabdeckung und Teststabilität

Die Testdaten liegen in zwei Formen vor: (a) eine Aggregation von Pass/Fail-Zählwerten je Branch in `evaluation/deep_code_analysis.json` sowie (b) Jest-JSON-Zusammenfassungen je Branch in `evaluation/data_safe/test_results.json`. In `evaluation/deep_code_analysis.json` reichen die pro Branch gezählten `test_pass`-Werte von 0 bis 14 und die `test_fail`-Werte von 9 bis 37. 7 von 18 Branches weisen `test_pass=0` aus; 11 von 18 Branches weisen mindestens einen bestandenen Test aus. Die Jest-JSON-Ausgaben bestätigen dieses Bild in Form von Summen (z. B. `numFailedTestSuites`, `numFailedTests`, `numPassedTests`); eine normalisierte Kategorisierung der Fehlertypen (z. B. nach “Auth/HTTP-Status”, “Schema”, “Datenpersistenz”) ist in den Artefakten nicht enthalten.

Für die Aufgabenfortschritts-Metrik wird zusätzlich `evaluation/task_pipeline_v3.json` herangezogen. Für die dort ausgewiesene Test-Validierung (`verified`) ergeben sich über alle Branches: T1 ist in 10/18 Branches verifiziert, T2 in 4/18, T4 in 0/18 und T5 in 0/18. Für T5 wird in `evaluation/task_pipeline_v3.json` außerdem `implemented=0/18` ausgewiesen. Bei T3 sind in `evaluation/task_pipeline_v3.json`

`verified=13/18` und `implemented=7/18` vermerkt; diese Differenz wird in dieser Arbeit als Messartefakt der verwendeten “implemented”-Heuristik ausgewiesen (die übergreifende Hierarchie $Verified \subseteq Implemented$ wird über die in Abschnitt 6.2 festgelegte Implemented-Definition hergestellt).

6.4.3 Architektur- und Strukturindikatoren

Als strukturbezogene Indikatoren werden in `evaluation/deep_code_analysis.json` für einzelne Aufgaben Implementationsklassen sowie Persistenzvarianten ausgewiesen. Für T1 (Persistenz) werden unterschiedliche Backends beobachtet: 14/18 Branches sind als *sqlite* klassifiziert, jeweils 1/18 als *mongo*, *postgres*, *json-file* und *memory/other*. Für T2 (Auth) werden 17/18 Branches als *implemented* und 1/18 als *none* klassifiziert. Für T3 (Kategorien) sind 7/18 Branches als *implemented* und 11/18 als *none* klassifiziert; für T4 (Attachments) 8/18 als *implemented* und 10/18 als *none*. Für T5 (Kalendar) ist in `evaluation/deep_code_analysis.json` über alle 18 Branches hinweg *none* ausgewiesen, konsistent zu `evaluation/task_pipeline_v3.json` (`implemented=0/18`, `verified=0/18`).

Die Codeumfangs-Metrik `tsFileCount` (`evaluation/code_quality_metrics.json`) liegt über alle Branches zwischen 43 und 80 TypeScript-Dateien (Median: 54) und liefert damit einen groben, rein quantitativen Indikator für Codebasis-Variation.

6.4.4 Zusammenhänge mit Task-Fortschritt

Für eine rein deskriptive Gegenüberstellung wird die Anzahl als `implemented` markierter Tasks pro Branch aus `evaluation/task_pipeline_v3.json` genutzt. Die Branches verteilen sich dabei auf 1 bis 4 als `implemented` markierte Tasks (T5 ist in allen Fällen nicht implementiert). In den gruppierten Mittelwerten ko-aufreten höhere Werte von `anyUsage` und `consoleUsage` mit höherer `implemented`-Taskanzahl (z.B. `anyUsage` im Mittel 8.8 bei 2 implementierten Tasks vs. 12.4 bei 3 vs. 16.7 bei 4). Analog zeigt die Gegenüberstellung nach ESLint-Fehleranzahl (0 vs. 1 Fehler laut `evaluation/data_safe/lint_results.json`) Unterschiede in den Medianen von `consoleUsage` (4.5 vs. 7.5) und `anyUsage` (7.5 vs. 9); diese Darstellung ist als kleine Fallzahl bei 0 Fehlern (4 Branches) zu lesen.

Darüber hinaus enthalten die vorliegenden Artefakte keine Kennzahlen zu Testabdeckung in Prozent, keine Komplexitätsmaße (z.B. zyklomatische Komplexität), keine TypeScript-Compilerfehlerstatistiken, keine Performance-/Bundle-Metriken und keine konsolidierten Frontend-Lighthouse-Ergebnisse. Aus diesem Grund werden hier keine weitergehenden, metrisch gestützten Aussagen über Abdeckung, Komplexität oder Laufzeitverhalten getroffen.

6.4.5 Kurzfazit: Beobachtete Qualitätsmerkmale

- ESLint: 4/18 Branches ohne ESLint-Fehler, 14/18 mit genau einem Fehler (keine Regelkategorien verfügbar).

- Typisierung/Debug-Surrogate: `anyUsage` variiert stark (2–42), `consoleUsage` liegt zwischen 4 und 19.
- Tests: Testausgänge sind heterogen (`test_pass` 0–14; `test_fail` 9–37); 7/18 Branches ohne bestandene Tests.
- Aufgabenvalidierung (Pipeline v3): T1 ist in 10/18 Branches verifiziert, T2 in 4/18; T4 und T5 in 0/18.
- Architekturindikatoren: Persistenz- und Feature-Varianten sind sichtbar (mehrere Persistenztypen; T2 überwiegend implementiert; T5 in allen Branches nicht implementiert).

6.5 Chat-Interaktionsmuster

Dieses Unterkapitel beschreibt beobachtbare Muster der Interaktion zwischen den Teilnehmenden (Teilnehmer A–R) und dem im Experiment eingesetzten Tool *GitHub Copilot* mit dem zugrundeliegenden LLM *Claude Sonnet 4.5*. Als Datenbasis dienen ausschließlich die 18 Chat-Exports in `thesis/chats/` (.docx) sowie der Task-Status aus `evaluation/task_pipeline_v3.json`. Ein separates, maschinell generiertes Chat-Metrik-Artefakt liegt nicht vor; alle berichteten Metriken wurden direkt aus den Chat-Exports extrahiert. Die Zuordnung Chat ↔ Branch erfolgt über die im Evaluationskapitel beschriebene Normalisierung der Chat-Dateinamen und deren Mapping auf Branches. Alle Aussagen werden als Zählwerte, Spannweiten oder Verteilungskennwerte berichtet; eine qualitative Inhaltsbewertung der Antworten erfolgt in diesem Unterkapitel nicht.

6.5.1 Umfang und Intensität der Interaktion

Messdefinition. Für alle 18 Chats wurde die Wortanzahl als whitespace-tokenisierte Wortzählung auf dem aus .docx extrahierten Text bestimmt. Wortanzahl, Zyklenzahl und Anzahl der Nutzerbeiträge sind in diesem Abschnitt als Intensitätsindikatoren operationalisiert und werden nicht als Leistungs- oder Effizienzmaße interpretiert.

Ergebnis. Die Wortanzahl pro Chat liegt zwischen 4046 und 26833 Wörtern (Median: 7137.5; Quartile: Q1=5330.5, Q3=8862.25).

Messdefinition. Die Anzahl der Prompt-Antwort-Zyklen wurde als Sequenzpaar aus einem Nutzerbeitrag (in den Exports typischerweise markiert durch `You said:` oder einen Namenspräfix `X:`) und der unmittelbar folgenden Modellantwort (in den Exports u. a. markiert durch `GitHub Copilot:` oder `ChatGPT said:`) gezählt. Die genannten Sprecherkennzeichnungen werden ausschließlich zur Segmentierung in Nutzerbeiträge und Modellantworten verwendet.

Ergebnis. Über alle Chats liegt die Anzahl der Prompt-Antwort-Zyklen zwischen 12 und 45 (Median: 16.5; Quartile: Q1=14.25, Q3=19.75).

Messdefinition. Als zusätzlicher Intensitätsindikator wurde die Anzahl der erkannten Nutzerbeiträge pro Chat bestimmt.

Ergebnis. Diese liegt zwischen 53 und 777 Nutzerbeiträgen (Median: 144). Damit werden in den Chat-Exports sowohl Fälle mit wenigen Dutzend Nutzerbeiträgen als auch Fälle mit mehreren hundert Nutzerbeiträgen beobachtet.

6.5.2 Struktur der Prompts

Messdefinition. Für die formale Beschreibung der Prompt-Struktur wurden alle 3390 erkannten Nutzerbeiträge aggregiert ausgewertet. Die folgenden Merkmale wurden regelbasiert bestimmt; die Kategorien sind nicht disjunkt (ein Nutzerbeitrag kann mehrere Merkmale gleichzeitig aufweisen).

Ergebnis. Ein-Satz-Prompts (operationalisiert als Nutzerbeiträge mit höchstens einem Satzendzeichen .?!) treten in 1901 von 3390 Fällen auf (56.1%). Strukturierte Mehrpunkt-Prompts (operationalisiert über Aufzählungs-/Schrittmuster, z.B. • oder nummerierte Listen sowie Schlüsselwörter wie *To do/Schritt*) treten in 471 von 3390 Fällen auf (13.9%).

Kontextangaben (operationalisiert über Kontextmarker wie *repo*, *backend*, *frontend*, *docker* oder *workspace*) treten in 1555 von 3390 Fällen auf (45.9%). Explizite Anforderungen/Constraints (operationalisiert über modale/negierende Marker wie *muss/must/do not/ausschließlich*) treten in 307 von 3390 Fällen auf (9.1%). Schrittanweisungen (operationalisiert über Marker wie *Schritt/first/zu*erst oder nummerierte Sequenzen) treten in 382 von 3390 Fällen auf (11.3%).

Nachforderungen bzw. erneute Anweisungen (operationalisiert über Wiederholungs-/Umformulierungsmarker wie *nochmal/anders/rewrite*) treten in 67 von 3390 Fällen auf (2.0%).

6.5.3 Debugging-Interaktionen und Iterationsmuster

Messdefinition. Debugging-bezogene Nutzerbeiträge wurden über Vorkommen typischer Fehlermarker (z.B. *error*, *exception*, *failed*, *TypeError*, *TS...*, *eslint*) erfasst. Die folgenden Iterations- und Wiederholungsmetriken beschreiben ausschließlich Textmuster in den Nutzerbeiträgen; daraus werden keine Aussagen über Emotionen, Kompetenz oder Motivation abgeleitet.

Ergebnis. Insgesamt enthalten 296 von 3390 Nutzerbeiträgen solche Fehlermarker (8.7%).

Messdefinition. Als *Iteration* wurde eine Folge von Nutzerbeiträgen operationalisiert, die jeweils Fehlermarker enthalten (aufeinanderfolgende Korrekturversuche im selben Chatabschnitt).

Ergebnis. Über alle Chats hinweg wurden 254 solche Iterationssequenzen gezählt. Die maximale Länge aufeinanderfolgender Fehlermarker-Nutzerbeiträge in einem einzelnen Chat liegt zwischen 1 und 9 (Median: 2; Maximum: 9).

Messdefinition. Als einfache Form von *Wiederholung* wurde zusätzlich erfasst, ob zwei aufeinanderfolgende Fehlermarker-Nutzerbeiträge eine identische normalisierte FehlerSignatur (erste erkannte Error-/Exception-Zeile) enthalten.

Ergebnis. Über alle Chats hinweg wurden 23 unmittelbare Wiederholungen identischer Fehlersignaturen gezählt.

Messdefinition. Als Abbruch-/Wechselindikator wurden Task-Sprünge in den Nutzerbeiträgen über Task-Mentions (Task 1-8, Aufgabe 1-8, T1-T8) erfasst. Task-Wechsel werden in diesem Unterkapitel als Strukturmerkmal der Interaktion berichtet und nicht bewertet.

Ergebnis. 148 von 3390 Nutzerbeiträgen enthalten mindestens eine Task-Mention (4.4%). In 8 von 18 Chats wird mindestens ein Wechsel der zuerst genannten Task-Nummer beobachtet; über alle Chats hinweg wurden 74 solche Task-Wechsel gezählt.

6.5.4 Modelltypische Antwortmuster (Claude Sonnet 4.5)

Messdefinition. Für die Modellantworten wurden 335 erkannte Antworten aggregiert ausgewertet. Als Modellantworten werden Textsegmente erfasst, die in den Exports als Antwort des Assistenzsystems markiert sind (z.B. durch `GitHub Copilot:` oder `ChatGPT said:`); diese Kennzeichnungen werden ausschließlich zur Segmentierung verwendet.

Ergebnis. Codeblöcke in der Modellantwort (operationalisiert über das Vorkommen von Markdown-Fences “”) treten in 122 von 335 Modellantworten auf (36.4%). Rückfragen bzw. Frageformen (operationalisiert über ein abschließendes Fragezeichen oder Formulierungen wie `Would you like/Can you`) treten in 21 von 335 Modellantworten auf (6.3%).

Tooling-/Aktionsformulierungen (operationalisiert über Marker wie `Ran terminal command` oder `Replace String in File`) treten in 41 von 335 Modellantworten auf (12.2%). Architektur-/Strukturbegriffe (operationalisiert über Schlüsselwörter wie `controller`, `service`, `repository`, `routes`, `docker`, `sqlite`) treten in 152 von 335 Modellantworten auf (45.4%).

6.5.5 Beziehung zwischen Chat-Mustern und Task-Fortschritt (deskriptiv)

Messdefinition. Für eine rein deskriptive Gegenüberstellung wird der Task-Fortschritt je Branch aus `evaluation/task_pipeline_v3.json` (Anzahl `implemented` und `verified` Tasks) neben die Interaktionsmetriken gestellt. Die folgenden Aussagen beschreiben Ko-Vorkommen innerhalb derselben Branches; es werden keine Wirk- oder Kausalzusammenhänge abgeleitet.

Ergebnis. Über die 18 Branches hinweg liegen die `implemented`-Taskanzahlen zwischen 1 und 4 (T5 ist in `evaluation/task_pipeline_v3.json` in allen Branches nicht implementiert).

In der Gruppe mit 2 implementierten Tasks (n=6) liegt die Wortanzahl zwischen 4046 und 9190 (Median: 6449) und die Zyklenzahl zwischen 12 und 20 (Median: 14.5); die Anzahl verifizierter Tasks liegt zwischen 0 und 2 (Median: 2). In der Gruppe mit 3 implementierten Tasks (n=8) liegt die Wortanzahl zwischen 4863 und 26833 (Median:

8865) und die Zyklenzahl zwischen 16 und 45 (Median: 18); die Anzahl verifizierter Tasks liegt zwischen 0 und 2 (Median: 2). In der Gruppe mit 4 implementierten Tasks ($n=3$) liegt die Wortanzahl zwischen 4764 und 6663 (Median: 4919) und die Zyklenzahl zwischen 14 und 21 (Median: 14); die Anzahl verifizierter Tasks liegt zwischen 0 und 2 (Median: 2). Die Kombination aus hoher Wortanzahl (bis 26833) und hoher Zyklenzahl (bis 45) tritt innerhalb derselben Gruppe (3 implementierte Tasks) auf.

Dieses Unterkapitel beschreibt ausschließlich beobachtbare Interaktionsmuster und deren Messwerte. Es enthält keine Effizienz- oder Leistungsbewertung, keine Kompetenzzuschreibung, keine psychologischen Deutungen und keine Modellbewertung. Die Interpretation dieser Muster sowie deren Einordnung erfolgt ausschließlich in Kapitel 7.

6.6 Triangulation der Datenquellen

Dieses Unterkapitel stellt die in Abschnitt 6.2 bis 6.5 berichteten Ergebnisse sowie Survey-Merkmale nebeneinander. Ziel ist eine rein deskriptive Triangulation auf Ebene der 18 Branches (Teilnehmer A–R): Pipeline-Fortschritt (`attempted/implemented/verified`), beobachtete Strukturindikatoren der Implementationen, technische Metriken, Interaktionsmetriken aus den Chat-Exports sowie Survey-Angaben. Alle Aussagen werden als Zählwerte, Spannweiten oder Verteilungskennwerte berichtet; daraus werden keine Wirk- oder Kausalzusammenhänge abgeleitet. Die folgenden Abschnitte ordnen diese Datenquellen paarweise und gruppenweise nebeneinander, um Muster und Spannweiten sichtbar zu machen.

6.6.1 Datenbasis und Aggregationslogik

Messdefinition. Die Triangulation erfolgt pro Branch als gemeinsame Analyseinheit. Der Pipeline-Fortschritt wird aus `evaluation/task_pipeline_v3.json` als Anzahl der pro Branch als `implemented` bzw. `verified` markierten Tasks (T1–T5) abgeleitet. Als ergänzende Strukturindikatoren (z.B. Persistenzvariante) und Test-Summen (`test_pass/test_fail`) wird `evaluation/deep_code_analysis.json` genutzt; technische Metriken wie `tsFileCount`, `anyUsage` und `consoleUsage` stammen aus `evaluation/code_quality_metrics.json`. Survey-Merkmale (Kompetenzgruppe und Hürdenkategorie) werden aus `thesis/surveys/Probanden-Einstufungsformular(Responses).xlsx` über das Mapping `evaluation/survey_mapping.json` übernommen. Chat-bezogene Metriken werden in diesem Unterkapitel nicht neu berechnet, sondern als die in Abschnitt 6.5 berichteten Werte (Extraktion aus `thesis/chats/`) übernommen.

Ergebnis. Die Branches verteilen sich auf 1 bis 4 als `implemented` markierte Tasks (Verteilung: 1 → 1 Branch, 2 → 6 Branches, 3 → 8 Branches, 4 → 3 Branches). Für `verified` ergeben sich 0 bis 2 verifizierte Tasks je Branch (Verteilung: 0 → 4 Branches, 1 → 1 Branch, 2 → 13 Branches). T5 ist in `evaluation/task_pipeline_v3.json` in allen Branches nicht implementiert und nicht verifiziert.

6.6.2 Pipeline-Fortschritt und Implementationsstrategien (Strukturindikatoren)

Messdefinition. Als Stellvertreter für ausgewählte Implementationsstrategien werden Strukturindikatoren aus `evaluation/deep_code_analysis.json` herangezogen. Für T1 betrifft dies die klassifizierte Persistenzvariante (z. B. *sqlite*, *mongo*); für T2–T4 werden Implementationsklassen (`implemented/none`) berichtet.

Ergebnis. Über alle Branches hinweg dominiert bei T1 *sqlite* (14/18); daneben treten *mongo* (1/18), *postgres* (1/18), *json-file* (1/18) und *memory/other* (1/18) auf. Bei Gruppierung nach `implemented`-Taskanzahl zeigt sich folgende Verteilung der Persistenzvarianten: bei 2 implementierten Tasks (n=6) *sqlite* in 5/6 Fällen und *postgres* in 1/6; bei 3 implementierten Tasks (n=8) *sqlite* in 7/8 und *mongo* in 1/8; bei 4 implementierten Tasks (n=3) *sqlite* in 2/3 und *json-file* in 1/3; der einzelne Branch mit 1 implementiertem Task ist *memory/other*.

Für T2 wird in `evaluation/deep_code_analysis.json` in 17/18 Branches `implemented` ausgewiesen; konsistent dazu liegen in `evaluation/task_pipeline_v3.json` `implemented` für T2 in 17/18 Branches vor. Für T3 sind 7/18 Branches in `evaluation/task_pipeline_v3.json` als `implemented` markiert; gleichzeitig ist T3 in 13/18 Branches `verified`. Für T4 sind 8/18 Branches als `implemented` markiert; `verified` ist bei T4 in allen Branches 0/18.

6.6.3 Pipeline-Fortschritt und technische Metriken

Messdefinition. Für eine deskriptive Gegenüberstellung werden pro Branch technische Metriken aus `evaluation/code_quality_metrics.json` (`tsFileCount`, `anyUsage`, `consoleUsage`) sowie Test-Summen aus `evaluation/deep_code_analysis.json` (`test_pass`, `test_fail`) neben die Anzahl `implemented` Tasks gestellt.

Ergebnis. Über alle 18 Branches liegt `tsFileCount` zwischen 43 und 80 Dateien; `anyUsage` liegt zwischen 2 und 42 und `consoleUsage` zwischen 4 und 19. Gruppiert nach `implemented`-Taskanzahl ergeben sich folgende Spannweiten und Mediane:

- 1 implementierter Task (n=1): `anyUsage`=6, `consoleUsage`=6, `tsFileCount`=43; `test_pass`=14, `test_fail`=12.
- 2 implementierte Tasks (n=6): `anyUsage` 2–30 (Median: 4), `consoleUsage` 4–17 (Median: 4.5), `tsFileCount` 43–63 (Median: 56.5); `test_pass` 0–14 (Median: 12), `test_fail` 9–37 (Median: 11).
- 3 implementierte Tasks (n=8): `anyUsage` 3–42 (Median: 8.5), `consoleUsage` 4–19 (Median: 10), `tsFileCount` 49–80 (Median: 54); `test_pass` 0–12 (Median: 0), `test_fail` 12–28 (Median: 15.5).
- 4 implementierte Tasks (n=3): `anyUsage` 15–19 (Median: 16), `consoleUsage` 4–9 (Median: 6), `tsFileCount` 47–74 (Median: 57); `test_pass` 0–13 (Median: 12), `test_fail` 10–13 (Median: 12).

Diese Gegenüberstellung beschreibt ausschließlich gemeinsame Verteilungen innerhalb derselben Branches.

6.6.4 Pipeline-Fortschritt und Chat-Interaktionsmetriken

Messdefinition. Als Chat-Metriken werden die in Abschnitt 6.5 definierten und berichteten Intensitäts- und Interaktionsindikatoren (Wortanzahl, Prompt-Antwort-Zyklen, Nutzerbeiträge, Fehlermarker-Iterationen) herangezogen. Für die Triangulation werden diese Werte neben die **implemented**- und **verified**-Taskanzahl aus `evaluation/task_pipeline_v3.json` gestellt.

Ergebnis. In Abschnitt 6.5 wird bereits eine gruppierte Gegenüberstellung nach **implemented**-Taskanzahl berichtet. Dort liegen (a) in der Gruppe mit 2 implementierten Tasks ($n=6$) Wortanzahl und Zyklenzahl zwischen 4046–9190 Wörtern (Median: 6449) bzw. 12–20 Zyklen (Median: 14.5), (b) in der Gruppe mit 3 implementierten Tasks ($n=8$) zwischen 4863–26833 Wörtern (Median: 8865) bzw. 16–45 Zyklen (Median: 18) und (c) in der Gruppe mit 4 implementierten Tasks ($n=3$) zwischen 4764–6663 Wörtern (Median: 4919) bzw. 14–21 Zyklen (Median: 14). Über alle Gruppen hinweg liegt die verifizierte Taskanzahl in `evaluation/task_pipeline_v3.json` je Branch zwischen 0 und 2.

6.6.5 Pipeline-Fortschritt und Survey-Merkmale

Messdefinition. Aus dem Survey werden (a) die Kompetenzgruppe (No-Code, Low-Code, Mid-Code, High-Coder) und (b) eine primäre Hürdenkategorie (Survey-Feld D) als kategoriale Merkmale herangezogen. Beide Merkmale werden ausschließlich als Gruppierungs- bzw. Häufigkeitsvariablen genutzt.

Ergebnis. Die Kompetenzgruppen verteilen sich auf No-Code (5/18), Low-Code (4/18), Mid-Code (5/18) und High-Coder (4/18). Tabelle 6.3 stellt diese Gruppen der **implemented**-Taskanzahl gegenüber.

Kompetenzgruppe	1 impl.	2 impl.	3 impl.	4 impl.
No-Code ($n=5$)	0	2	2	1
Low-Code ($n=4$)	1	1	2	0
Mid-Code ($n=5$)	0	1	2	2
High-Coder ($n=4$)	0	2	2	0

Tabelle 6.3: Kompetenzgruppe (Survey) vs. Anzahl **implemented** Tasks pro Branch (`evaluation/task_pipeline_v3.json`, $N = 18$).

Als primäre Hürdenkategorien (Survey-Feld D) treten am häufigsten Zeitdruck (5/18) und Debugging von Fehlern (5/18) auf, gefolgt von Aufgabenverständnis (3/18). In der Gruppe mit 2 implementierten Tasks ($n=6$) ist Zeitdruck 2/6-mal und Debugging 1/6-mal als primäre Hürde genannt; in der Gruppe mit 3 implementierten Tasks ($n=8$) Debugging 3/8-mal und Zeitdruck 2/8-mal. In der Gruppe mit 4 implementierten Tasks ($n=3$) treten Zeitdruck, Debugging und Aufgabenverständnis jeweils in 1/3 Fällen als primäre Hürde auf.

6.6.6 Abgrenzung zu Kapitel 7

Dieses Unterkapitel stellt ausschließlich Ko-Vorkommen und Verteilungen über mehrere Datenquellen hinweg dar. Es enthält keine Leistungs- oder Effizienzbewertung, keine Kompetenzzuschreibung und keine kausale Interpretation. Die Einordnung dieser Muster sowie die Diskussion möglicher Erklärungen erfolgt ausschließlich in Kapitel 7.

6.7 Zusammenfassung der Ergebnisse

Kapitel 6 dokumentiert die Ergebnisse der Evaluation und stellt beobachtete Ausprägungen und Verteilungen dar, ohne diese zu bewerten oder zu erklären. Die Ergebnisdarstellung basiert auf mehreren Datenquellen (Pipeline-Status `attempted/implemented/verified`, Code- und Test-Artefakte, Chat-Exports sowie Survey-Merkmale) und bleibt durchgängig deskriptiv. Im Folgenden werden die zentralen Befundlinien systematisch zusammengeführt.

Pipeline-Ergebnisse entlang der Tasks (T1–T5). Über die Tasks hinweg zeigt sich ein gemeinsamer Trichter-Effekt: Attempted-Markierungen sind in der Breite vorhanden, während die Anteile der als Implemented bzw. Verified ausgewiesenen Ergebnisse mit zunehmendem Integrations- und Abhängigkeitsgrad abnehmen. Dabei ist die Ausprägung dieses Trichters nicht in allen Branches gleichförmig, sondern zeigt Spannweiten zwischen sehr kurzem Task-Fortschritt und einem deutlich weiter reichenden Umsetzungsstand. In der Gegenüberstellung der Aufgaben treten dabei zwei Bündel hervor:

- **Backend-nahe Tasks (T1/T2).** T1 ist in den Branches in unterschiedlichen Persistenzvarianten umgesetzt, wobei eine Variante dominiert und mehrere Alternativen vereinzelt auftreten. T2 wird in der Mehrzahl der Branches als Implemented ausgewiesen und ist damit ein häufig realisiertes Backend-Element.
- **Integrations- und Frontend-nahe Tasks (T3–T5).** Bei T3 und T4 nehmen die Implemented-Ausweise ab; zugleich werden bei einzelnen Tasks Diskrepanzen zwischen Implemented und Verified sichtbar (ohne dass daraus in Kapitel 6 Schlussfolgerungen abgeleitet werden). T5 bleibt in den Artefakten durchgängig ohne Implemented- und ohne Verified-Ausweise.

Über alle Tasks hinweg gilt: Verified, Implemented und Attempted sind empirisch unterscheidbare Statuskategorien und fallen nicht notwendigerweise zusammen. Kapitel 6 beschreibt diese Statusausweise als Ergebnisvariablen der Pipeline und nutzt sie als gemeinsame Referenz, um Code-, Test- und Chat-Artefakte pro Branch in Beziehung zu setzen.

Technische Qualität (aggregiert, ohne Kausalannahmen). Die in Kapitel 6 berichteten Qualitäts- und Technikindikatoren zeigen insgesamt eine heterogene Ausprägung über die Branches hinweg. Dazu gehören (a) Unterschiede im Linting-Status und in

der Häufigkeit typischer TypeScript-Surrogate (z. B. Any-Nutzung), (b) Spannweiten bei strukturbezogenen Metriken (z. B. Anzahl TypeScript-Dateien) sowie (c) variierende Testausgänge bis hin zu Konstellationen mit Fehlanteilen. Diese Befunde werden in Kapitel 6 als Ko-Existenz von Pipeline-Fortschritt und technischen Merkmalen beschrieben; es wird keine Korrelation oder Wirkbeziehung behauptet. Zusätzlich wird sichtbar, dass sich technische Ausprägungen nicht auf einen einheitlichen "Projektzustand" reduzieren lassen: Branches mit ähnlichem Pipeline-Fortschritt können unterschiedliche Muster in Linting-, Typisierung- und Testindikatoren aufweisen. Zugleich werden die Messgrenzen betont: Die verwendeten Indikatoren ersetzen keine Coverage-, Komplexitäts- oder Performance-Metriken und erlauben entsprechend keine Aussagen zu nicht gemessenen Qualitätsdimensionen.

Chat-Interaktionsmuster (kompakt). Die Chat-Analysen zeigen große Spannweiten bei der Interaktionsintensität, insbesondere bei Wortanzahl und Prompt-Antwort-Zyklen. Über die Branches hinweg koexistieren kurze, wenig strukturierte Prompts mit detaillierten Mehrpunkt-Prompts; beide Formen treten im selben Datensatz nebeneinander auf. Als wiederkehrendes Strukturmerkmal sind Debugging-Iterationen sichtbar (z. B. wiederholte Fehlermarker und erneute Anpassungsversuche), ohne dass daraus eine qualitative Einordnung ("gut/schlecht") abgeleitet wird. Neben Intensitätsmaßen werden in Kapitel 6 auch Muster in der Interaktionsstruktur beschrieben, etwa wiederkehrende Task-Referenzen, Wechsel zwischen Aufgaben sowie Tooling-/Aktionsformulierungen im Verlauf der Chats.

Triangulation auf Meta-Ebene. Die Triangulation macht sichtbar, dass unterschiedliche Datenquellen teils kongruente, teils divergente Signale liefern: Pipeline-Status, technische Indikatoren, Testausgänge, Chat-Merkmale und Survey-Merkmale können für dieselben Branches zusammenpassen oder auseinanderlaufen. Insbesondere verdeutlicht die Gegenüberstellung, dass *Verified* $\neq$ *Implemented* $\neq$ *Attempted* und dass Spannungsfelder zwischen Statusausweisen, technischen Beobachtungen und Interaktionsmustern empirisch auftreten. Auch Survey-Merkmale werden als kategoriale Kontextvariablen einbezogen (Selbsteinstufungsgruppen und benannte Hürdenkategorien), ohne dass daraus individuelle Zuschreibungen oder Bewertungen abgeleitet werden. In der Gesamtschau entstehen damit nebeneinander stehende Ergebnislinien, die Gemeinsamkeiten und Unterschiede zwischen Branches sichtbar machen, ohne diese zu erklären. Kapitel 6 stellt diese Spannungsfelder dar, löst sie jedoch nicht auf.

Abgrenzung zu Kapitel 7. Kapitel 6 beschreibt, *was* beobachtet wurde; Kapitel 7 diskutiert, *wie* diese Befunde einzuordnen sind und *warum* bestimmte Muster auftreten könnten.

Literatur

- Anderson, John R. (1996). *The Architecture of Cognition*. Mahwah, NJ: Psychology Press.
- Barke, Shraddha, Michael B. James und Nadia Polikarpova (2023). „Grounded Copilot: How Programmers Interact with Code-Generating Models“. In: *Proceedings of the ACM on Programming Languages* 7.OOPSLA1, S. 1–27. DOI: 10.1145/3586030.
- Bavarian, Mohammad u. a. (2022). „Efficient Training of Language Models to Fill in the Middle“. In: *arXiv preprint arXiv:2207.14255*. DOI: 10.48550/arXiv.2207.14255.
- Brandtner, Matthias, Philipp Leitner und Cor-Paul Bezemer (2024). „Challenges of Brownfield Software Modernization in Practice: A Systematic Review“. In: *Journal of Systems and Software*. DOI: 10.1016/j.jss.2024.111743.
- Collins, Allan, John Seely Brown und Susan E. Newman (1989). „Cognitive Apprenticeship: Teaching the Crafts of Reading, Writing, and Mathematics“. In: *Knowing, Learning, and Instruction: Essays in Honor of Robert Glaser*, S. 453–494.
- Cook, Thomas D. und Donald T. Campbell (1979). *Quasi-Experimentation: Design & Analysis Issues for Field Settings*. Boston: Houghton Mifflin. ISBN: 978-0395307878.
- Dreyfus, Hubert L. und Stuart E. Dreyfus (1986). *Mind over Machine: The Power of Human Intuition and Expertise in the Era of the Computer*. New York: Free Press.
- Forsgren, Nicole, Margaret-Anne Storey und Thomas Zimmermann (2021). „SPACE: A Framework for Understanding Productivity in Software Engineering“. In: *Communications of the ACM* 64.6, S. 33–36. DOI: 10.1145/3454122.
- Green, Thomas R. G. (1989). *Cognitive Dimensions of Notations*. Cambridge University Press, S. 443–460.
- Horvitz, Eric (1999). „Principles of Mixed-Initiative User Interfaces“. In: *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, S. 159–166. DOI: 10.1145/302979.303029.
- ISO/IEC 25010:2011 *Systems and Software Engineering—System and Software Quality Models* (2011). Standard.
- Kahneman, Daniel (2011). *Thinking, Fast and Slow*. New York: Farrar, Straus und Giroux. ISBN: 978-0374275631.
- Lave, Jean und Etienne Wenger (1991). *Situated Learning: Legitimate Peripheral Participation*. Cambridge: Cambridge University Press.
- Lee, John D. und Katrina A. See (2004). „Trust in Automation: Designing for Appropriate Reliance“. In: *Human Factors* 46.1, S. 50–80. DOI: 10.1518/hfes.46.1.50.
- Martin, Robert C. (2009). *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River, NJ: Prentice Hall. ISBN: 978-0132350884.

- Mozannar, Hussein, Gagan Bansal, Adam Fourney und Eric Horvitz (2022). „Reading Between the Lines: Modeling User Behavior and Costs in AI-Assisted Programming“. In: *arXiv preprint arXiv:2210.14306*.
- Nadi, Sarah u. a. (2022). „Human-AI Collaboration in Software Engineering: Challenges and Opportunities“. In: *arXiv preprint arXiv:2206.01081*. DOI: 10.48550/arXiv.2206.01081.
- Parasuraman, Raja und Dietrich H. Manzey (2010). „Complacency and Bias in Human Interaction with Automation: An Attentional Integration“. In: *Human Factors* 52.3, S. 381–410. DOI: 10.1177/0018720810376055.
- Pearce, Henry u. a. (2022). „Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions“. In: *IEEE Symposium on Security and Privacy*. DOI: 10.1109/SP46214.2022.9833573.
- Peng, Sida, Eirini Kalliamvakou, Peter Cihon und Mert Demirel (2023). „The impact of AI on developer productivity: Evidence from GitHub Copilot“. In: *arXiv preprint arXiv:2302.06590*. DOI: 10.48550/arXiv.2302.06590.
- Saretsky, Gary (1972). „The OEO P.C. Experiment and the John Henry Effect“. In: *Phi Delta Kappan* 53.9, S. 579–581.
- Sweller, John (1988). „Cognitive Load During Problem Solving: Effects on Learning“. In: *Cognitive Science* 12.2, S. 257–285.
- Vaithilingam, Priyan u. a. (2022). „Expectations vs. Reality: Evaluating Code Completion Tools in Realistic Settings“. In: *CHI Conference on Human Factors in Computing Systems*. DOI: 10.1145/3491102.3502031.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser und Illia Polosukhin (2017). „Attention Is All You Need“. In: *Advances in Neural Information Processing Systems* 30, S. 5998–6008.
- Witte, Thomas u. a. (2023). „Context Windows and the Limits of AI-Assisted Programming“. In: *arXiv preprint arXiv:2308.10253*. DOI: 10.48550/arXiv.2308.10253.
- Wohlin, Claes, Per Runeson, Martin Höst, Magnus C Ohlsson, Björn Regnell und Anders Wesslén (2012). *Experimentation in Software Engineering*. Springer. ISBN: 978-3-642-29043-5. DOI: 10.1007/978-3-642-29044-2.